TABLE OF CONTENTS

목차

PORTFOLIO · 김단비 · BACKEND

01
COVER
커버 — Portfolio 메인

02
01 · EXPERIENCE
실무 경력 — 2년의 운영 오너십

03
02 · PROJECT
프로젝트 #1 — 다2조부르릉

04
03 · PROJECT
프로젝트 #2 — ants-camp

PORTFOLIO · 김단비 · BACKEND

운영에서 시작해 LLMOps까지 정합성·신뢰성을 설계하는 백엔드.

회원제 SaaS 운영팀에서 **RDBMS 마이그레이션·동시성 제어·다국가 확장**을 직접 풀어내고,
그 한계를 학습 동기로 **모놀리식 → MSA → LLMOps**까지 설계 경험을 단계적으로 확장했습니다.
"왜 이 기술을 쓰는가"를 끝까지 묻고, 운영 환경의 정합성·신뢰성 문제를 설계 단계에서 해결합니다.

● github.com/danbeekimm ● https://velog.io/@danbeekimm

WORKS

프로젝트 · 경력

세 작업을 시간순으로. **실무에서 출발해 모놀리식·MSA를 거쳐 LLMOps에 닿은 궤적** — 현장에서 마주친 한계가 학습 동기가 되고, 다음 프로젝트의 설계 결정으로 이어지는 흐름.

💼 EXPERIENCE · 실무 경력 2022.11 ~ 2025.03

SQL 의존성과 동시성을 운영 환경에서 풀어낸 2년

풀스택 개발자 — 회원제 특허 SaaS의 고도화·운영

100% MyBatis 환경의 **PostgreSQL → MSSQL 전면 마이그레이션**을 옵티마이저·라운드트립 단위로 재설계하고,
1만 건 분할 비동기 다운로드의 **이용량 우회 race condition**을 "임계구역 위치 재배치"로 풀었습니다.
여기서 체감한 한계가 그대로 부트캠프 학습으로 이어졌습니다.

동일 수준	원천 차단	3 → 1
벤더 변경 후 응답 성능 유지	다중 창 동시 요청 한도 우회	실무 한계 · 스터디 · 부트캠프 연결

`Spring` `MyBatis` `MSSQL · T-SQL UDF` `Stored Procedure` `비동기 동시성` `Apache POI`

담당: **6개 검색 사이트 유지보수·개선 — 대량 문서 뷰어 · 다운로드 · 비교보기**

상세 보기→

🚚 PROJECT #1 · 다2조부르릉 6인

정합성 보장과 장애 격리를 설계한 B2B 물류 MSA

12개 마이크로서비스 · DDD 4계층 헥사고날 · "검증=동기, 통지=비동기" 통신 분리

직전 모놀리식의 한계(한 도메인 결합이 전체 멈춤)를 학습 동기로 **MSA + DDD**로 확장.
분산 트랜잭션 없이 **Outbox 패턴**으로 이벤트 정합성을 보장하고, **OR-Tools VRPTW**로 B2B 시간창 SLA를 제약 조건으로 모델링했습니다.
비관적 락은 스터디에서 학습한 트레이드오프를 실제 적용한 사례.

at-least-once	5분 → 3초	DB-level
Outbox · 결과적 일관성	VRPTW 자동 배정 (100건 · 데모)	비관적 락 race 원천 차단

`Spring Cloud 2025` `Kafka · Outbox` `OpenFeign` `Resilience4j CB` `OR-Tools VRPTW` `QueryDSL` `PESSIMISTIC_WRITE`

담당: **company-service 전담 + DeliveryManager 도메인**

da2joburureung 상세 보기→

🤖 PROJECT #2 · ants-camp 5인

LLM을 정량 평가하고, 안전하게 운영하는 백엔드

모의투자 MSA + RAG 챗봇 + LLM 기반 AIOps 설계/구현

대다수 RAG가 답변 품질을 "느낌"으로 판단할 때, **LLM-as-a-Judge + Pairwise + Self-preference bias 차단**으로 모델 교체를 데이터로 결정했습니다. Prometheus·Loki·Claude를 묶어 **알림에 원인 분석과 복구 액션을 자동 부착**한 AIOps는 AI를 운영 시스템 그 자체로 녹여낸 사례입니다.

2.38 → 4.04	64.3% → 8.3%	< 1분	74% ↓
faithfulness 향상 · 모델 교체	환각률 감소	MTTD · 7종 alert 룰 차등	캐시 히트 응답 단축 · LLM 호출 스킵

`Spring AI 1.0` `pgvector` `LLM-as-a-Judge` `Pairwise` `Prometheus · Loki · Alertmanager` `HITL · HMAC-SHA256` `Kafka · Redis`

담당: **assistant + notification + monitoring 전담 설계/구현**

ants-camp 상세 보기→

SKILLS

기술 스택

	💼 실무 경험 (2년)	🎓 학습·프로젝트
LANGUAGE	Java 8 · JavaScript	Java 17 · TypeScript
BACKEND FW	Spring 4.x (Legacy)	**Spring Boot 3.5** · **Spring Cloud** (Gateway · Eureka · Config · OpenFeign) · **Spring AI 1.0** · Spring Security
FRONTEND	JSP · AngularJS · jQuery	React · TypeScript · Tailwind CSS
DATA ACCESS	MyBatis · Stored Procedure (T-SQL)	Spring Data JPA · QueryDSL
DATABASE	MSSQL · MySQL · PostgreSQL	PostgreSQL (+**PostGIS · pgvector**) · Redis
ARCHITECTURE	모놀리식 (대규모 운영)	**MSA · DDD · Hexagonal · Event-Driven**
MESSAGING / RESILIENCE	—	Apache Kafka · **Outbox Pattern** · Resilience4j (Circuit Breaker · Retry)
OBSERVABILITY	운영 로그 분석 · CS 패턴 추적	Prometheus · Grafana · Loki · Promtail · Alertmanager · Zipkin
AI / LLM	—	Spring AI · RAG · pgvector · **LLM-as-a-Judge** (Eval · Pairwise) · **LLM 기반 AIOps** · Human-in-the-Loop
OPTIMIZATION / PARSING	Apache POI (대용량 문서 파싱 · DRM 시그널 식별)	Google OR-Tools (VRPTW)
TEST	JUnit · Mockito (팀 도입 주도)	JUnit · Mockito
DEVOPS	—	Docker · Docker Compose · GitHub Actions

김단비 · 백엔드 개발자 — Portfolio v2026.05

← 포트폴리오 메인으로

01 · OVERVIEW · 실무 경력

"내 서비스다" 생각하고 이슈를 끝까지

회원제 특허 SaaS의 **6개 검색 사이트 유지보수·개선 담당**으로, **대량 문서 뷰어·엑셀 다운로드·비교보기**를 메인으로 담당했습니다. 운영하면서 마주친 **코드·기획·데이터의 불일치**를 그냥 넘어가지 않고 끝까지 추적해 해결하는 방식으로 일했습니다. "기능을 구현하는 사람"이 아니라 **"서비스의 상태를 책임지는 사람"**으로서 운영을 대했습니다.

> "이슈를 발견하면 넘어가지 않고, 보고하고, 끝까지 해결했다."
>
> 대량 데이터를 다루는 회원제 SaaS — 6개 검색 사이트의 유지보수·개선. 대량 문서 뷰어·엑셀 다운로드·비교보기를 메인으로, 그 외 사용자 편의성·로그 분석·외부 API 연동·DB 마이그레이션까지 담당 영역을 좁게 한정하지 않고 서비스 전체의 상태를 능동적으로 관리했습니다.
>
> ● 풀스택 개발자 ● 2022.11 ~ 2025.03

이슈를 그냥 넘기지 않는다

코드 동작이 어색하거나 데이터가 이상하면 **원인을 끝까지 추적**. 표면 증상이 아닌 근본 원인까지 들고 가서 해결.

기획 단계부터 함께 협업한다

단순 구현자가 아니라 **기획 자문 · 사용자 관점 의견 제시 · 코드 불일치 검토**까지. 기획팀과 활발히 소통하며 더 나은 방향을 같이 찾음.

운영을 내 것으로 본다

담당 기능만 보는 게 아니라 **서비스 전체의 상태**를 지켜봄. CS 문의·로그·이상 패턴을 능동적으로 모니터링.

★ MAIN ROLE

6개 검색 사이트 유지보수·개선

담당 기능: **대량 문서 뷰어(이지뷰어) · 엑셀 다운로드 · 비교보기**

★ 이지뷰어 도면-텍스트 동기화 작업이 연간 사용자 만족도 조사에서 2위로 선정됨

+ ADDITIONAL

유틸리티 · 편의성 개선

형광펜 · DRM 필터링 · 메인-서브 뷰어 연동 · 다운로드 한도 모니터링

+ ADDITIONAL

분석 · 연동 · 마이그레이션

로그 기반 사용자 행동 분석 · 외부 번역 API 연동 · PostgreSQL → MSSQL 마이그레이션

🛠 코드 수정 루틴 — 유지보수 환경에서 체득한 작업 원칙

STAGE 01

변경 전 — 코드만 보지 않는다

- ✓ **SVN history**로 "이 코드 왜 이렇게 짰지?" 과거 결정 맥락 추적
- ✓ **기획서·과거 CS 이슈** 확인 — *같은 문제 두 번 풀지 않기*
- ✓ DB 직접 조회로 **코드가 가정한 데이터**와 **실제 데이터**의 간극 확인

STAGE 02

변경 중 — 추측 대신 재현

- ✓ **버그 재현 시나리오부터 확정** — "이 조건에서 100% 재현"을 만든 뒤 작업 시작
- ✓ 디버거 step-by-step으로 흐름 검증 — *코드 읽고 추론하지 않음*

STAGE 03

변경 후 — 내 일이 끝나는 시점을 늦춘다

- ✓ **운영 로그 직접 확인** — 변경 직후 며칠간 이상 패턴 추적
- ✓ **관련 CS 인입 의식적으로 추적** — *"배포 = 끝"이 아니라 "배포 + 며칠 = 끝"*

일관된 원칙: *"맥락을 끝까지 추적한다 + 추측 대신 재현으로 확인한다 + 내 일이 끝나는 시점을 며칠 뒤로 미룬다." 이 셋이 재직 기간 동안 큰 운영 사고 없이 작업을 유지한 실질적 근거.*

02 · HOW I WORK

이슈를 발견·추적·해결한 방식

"내 담당 기능"의 경계를 좁게 두지 않았습니다. **운영 중 이상한 점 → 보고 → 원인 추적 → 해결**을 능동적으로 반복했고, 특히 **기획과 코드의 불일치**를 발견하면 기획팀과 직접 의논해 정정한 경험이 많습니다.

CASE #1

기획 단계부터 함께한 협업 — 자문 · 사용자 관점 · 코드 불일치 검토

단순 구현자가 아닌, 기획과 사용자 사이를 잇는 사람으로 일하기

기획 자문	기획 초기 단계에서 **"시스템적으로 가능한 범위 · 비용 · 우회 방법"**에 대한 자문을 요청받음. 받은 기획을 그대로 구현하는 게 아니라, "이 방향이 운영에 어떤 부담을 줄지"를 함께 검토.
사용자 관점	**실제 사용자가 되어 서비스를 써보며** 불편한 지점을 직접 체감. 기획에 적힌 흐름과 실제 사용 경험 사이의 간극을 발견하면 의견을 정리해서 공유. "사용자 입장에서 편리한가"를 기준으로 기획팀과 활발히 소통.
코드 불일치 검토	새 기획안이 나오면 받자마자 **"이 구조가 지금 코드와 맞나?"**를 먼저 확인. 기획팀이 알고 있는 시스템 모습과 실제 운영 중인 코드의 동작이 다른 경우를 정리해서 공유 — 어느 쪽을 정정할지 의논해서 결정.
결과	"막상 만들고 보니 의도와 다르다"는 재작업 비용을 사전에 차단하고, 기획팀도 시스템의 실제 모습을 더 정확히 알게 됨. 단순한 백엔드 구현자가 아닌, 기획·사용자·코드 사이를 잇는 사람으로 자리매김.

CASE #2

반복 CS 문의의 원인 분석 → 팀 논의 → 사전 차단 구현

문제의 단서를 제공하고, 결정된 방향을 끝까지 책임진 사례

상황	"문서 업로드 시 오류가 발생한다"는 사용자 문의가 반복 접수. 1차 대응은 CS팀이 "E-DRM 해제 후 재업로드"를 수동 안내하는 방식이었음.
팀 결정·할당	"수동 안내 대신 시스템 레벨에서 사전 차단하자"는 방향이 팀에서 결정됨. 해당 작업이 본인에게 할당됨.
원인 분석	반복되는 Exception 로그를 분석해, 사내 E-DRM 암호화 문서를 업로드할 때 Java 기반 문서 파서들이 던지는 **특정 Exception 패턴**이 존재함을 식별. E-DRM 암호화로 문서의 바이너리 구조 자체가 변형되어 파서가 정상 문서로 인식하지 못하기 때문이라, "무작위 에러가 아니라 일정한 원인이 있는 케이스"임을 확인.
구현·결과	업로드 단계에서 외부 분석 API에 넘기기 전에 **사전 파싱으로 시그널을 감지**해 "암호화 해제 후 재업로드" 안내 메시지를 노출. 파일 형식별로 도구를 분리해 **MS Office 계열은 Apache POI, HWP는 hwplib**으로 동일 패턴 적용. 일반 파싱 오류와 E-DRM 케이스를 명확히 분기 → 불명확한 Exception 화면이 명확한 안내로 바뀌고, **동일 원인의 CS 문의 재발 없음**.

03 · CONTRIBUTIONS CATALOG

운영 기여 한눈에 보기

메인 업무(**이지뷰어·엑셀 다운로드·비교보기**)부터 마이그레이션·동시성·UX·다국가 출시·팀 문화까지, 운영 환경의 다양한 문제를 직접 해결한 작업들의 카탈로그. ★ DEEP DIVE 배지가 있는 항목은 본문에서 자세히 다룸.

이지뷰어 ↔ 도면 팝업 실시간 동기화 ★ DEEP DIVE
사용자 만족도 ★ 2위
이지뷰어 문헌 이동 시 별도 팝업 도면을 자동 동기화. **URL 재호출 → 부분 렌더링(동기) → 비동기 + 콜백 재구성**의 3단계로 깜빡임·포커스 강탈·동기 멈춤을 차례로 제거. **연간 사용자 만족도 조사 2위**.
→ 07 · DEEP DIVE — VIEWER UX

PG → MSSQL 슬로우 쿼리 개선 ★ DEEP DIVE
MyBatis · T-SQL
MSSQL 2016에 `string_agg(DISTINCT)` 가 없어 STUFF + FOR XML PATH 우회 → 성능 저하. 서브쿼리·JOIN·WHERE 구조 조정 후 **임시 테이블 단계 분할**로 해결, 기존 환경 대비 동일 응답 성능 유지.
→ 04 · DEEP DIVE — DB MIGRATION

유료 다운로드 한도 우회 차단 동시성 ★ DEEP DIVE
1만 건 분할 비동기 처리에서 다중 창 동시 요청으로 한도 우회 가능했던 race condition을 **사전 차감**으로 해결. 락 기반 대안과 트레이드오프 비교 후 환경에 맞는 도구 선택.
→ 05 · DEEP DIVE — CONCURRENCY

대규모 특허 번역 — fan-out 설계 의사결정 ★ DEEP DIVE
UX 성능
200건 번역 노출 시 **서버 일괄 vs 클라이언트 fan-out**의 트레이드오프(인프라 부하↑ vs 체감 응답성↑)를 정리. 인프라 수용량·페이지 UX 중요도를 근거로 fan-out 채택. 로딩 저하 우려에도 **눈에 띄는 이슈 없이 출시**.
→ 06 · DEEP DIVE — STREAMING UX

E-DRM 암호화 문서 사전 차단 ★ DEEP DIVE
Apache POI · hwplib
반복 CS 문의의 원인 추적 → *E-DRM 암호화로 바이너리 구조가 변형되어 Java 파서가 일관된 Exception을 던지는 패턴*을 식별. 외부 API 호출 전 **Apache POI(MS Office) · hwplib(HWP)로 사전 파싱 검증**해 명확한 안내로 분기. **동일 사유 CS 재발 없음**.
→ 02 · HOW I WORK · CASE #2

EU 지역 서비스 신규 출시 다국가
검증된 타 국가 서비스의 구조·패턴을 분석해, **EU 특화 요구(언어·지역 정책)를 반영하면서도 일관성 있는 구조**로 안정적 출시. 기존 시스템의 추상화 레벨을 유지하며 확장한 사례.

JUnit/Mockito 테스트 도입 주도 팀 문화
인증이 필요한 컨트롤러 테스트 방법론을 정립해 팀 대상 발표 → 팀 표준 가이드라인 마련. 단순 적용이 아닌 "왜 이렇게 테스트해야 하는지"까지 공유.

비교 뷰어 스크롤 동기화 UX 알고리즘
기존 **절대 이동거리 동기화**는 길이가 다른 두 문서에서 어긋남 → **이동거리 비율(0~1) 동기화**로 변경해 양쪽 문서가 항상 같은 상대 위치를 보여줌. "UX 문제를 코드 구조로 푼" 사례.

형광펜 대규모 고도화 UX 데이터 모델
드래깅·리사이징 레이어 도입, 색상 16 → 40+개 확장, 저장 포맷 **XML → JSON** 전환. 데이터 모델 자체를 재설계해 향후 기능 확장을 쉽게 만든 작업.

로그 기반 사용자 행동 패턴 조사 운영 지원
CS·영업팀 요청을 받아 **로그인 시간이 불규칙한 사용자 등 의심 패턴 대상자를 로그 다각 분석으로 조사**해 운영팀에 보고. 단순 데이터 조회를 넘어 패턴 인식 관점으로 접근.

04 · DEEP DIVE — DB MIGRATION

DEEP DIVE — DB MIGRATION

PostgreSQL → MSSQL 마이그레이션 후 슬로우 쿼리 개선

시행착오로 해결한 작업을 이후 학습으로 원리 레벨에서 설명할 수 있게 된 사례

P · 팀 차원에서 PG → MSSQL(구버전, 2016) 전체 이관을 진행. 마이그레이션 후 **특정 쿼리가 눈에 띄게 느려진 케이스**를 담당해 개선. *특히 핵심 원인은, 중복 제거 문자열 집계에서 PG의* `string_agg(DISTINCT)` *가 MSSQL 2016에는 없어* ***STUFF + FOR XML PATH + DISTINCT 서브쿼리****로 우회해야 했고, 이 우회 자체가 큰 성능 저하를 유발.*

A · 같은 결과를 내는 여러 SQL 형태를 시도하며 단계적으로 접근:

① 호출 경로 식별 — 영향 범위 먼저 파악

슬로우 쿼리가 발생하는 화면·기능을 먼저 식별해 회귀 검증 범위를 가늠.

이유: *쿼리만 보고 고치면 같은 쿼리가 호출되는 다른 화면에서 회귀가 날 수 있음.*

② 서브쿼리 ↔ JOIN ↔ UNION 변환 시도

서브쿼리로 짠 부분을 JOIN으로, JOIN을 UNION으로 바꿔보며 MSSQL에서 어떤 형태가 빠른지 실행 시간 측정으로 비교.

결과: *일부 쿼리는 개선, 일부는 미미.*

③ WHERE절 구조 조정

조건 순서·표현 방식을 다양하게 시도.

결과: *미미한 개선. 근본 해결은 안 됨.*

④ 임시 테이블로 단계 분할 — 결정적 해결

복잡한 한 방 쿼리를 단계별로 쪼개서 임시 테이블에 넣고 다음 단계에서 사용.

결과: *큰 개선 확인.* ***이게 결정적이었음.***

R · 벤더가 바뀌어도 **기존 환경 대비 동일 수준의 응답 성능 유지**하며 슬로우 쿼리 개선 완료.

> 당시 한계와 이후 학습
>
> 당시엔 옵티마이저 개념을 깊이 알지 못한 채 시행착오로 해결. **이후 학습 결과로 이해한 메커니즘**: MSSQL 옵티마이저는 참여 테이블 수와 데이터 분포가 복잡해질수록 통계 추정이 빗나가 *잘못된 조인 순서나 비효율적 조인 방식(예: 해시 조인이 적절한데 중첩 루프를 택함)*을 선택하는 경향이 있음. 거대한 단일 쿼리를 통째로 던지면 옵티마이저가 길을 잃는 상황이었던 것. 임시 테이블 분할은 각 단계마다 데이터 크기·통계가 명확한 상태에서 옵티마이저가 판단하도록 강제하는 효과. **다음 작업 시 접근법**: 실행 계획을 먼저 확인하고 옵티마이저의 추정·실제 차이를 진단한 뒤 접근.

> → 학습 연결 고리
>
> 이 경험에서 **SQL 의존성·벤더 종속성의 한계**를 체감 → **ORM(JPA) 학습 동기**로 직결 → 영속성 컨텍스트·벤더 독립 데이터 접근 계층 설계를 학습 → 부트캠프 프로젝트의 **JPA + QueryDSL 설계**에 적용.

05 · DEEP DIVE — CONCURRENCY

DEEP DIVE — CONCURRENCY

임계구역을 "긴 로직 뒤"가 아닌 "맨 앞"으로 옮긴 비동기 동시성 설계

1만 건 분할 비동기 처리에서 다중 창 동시 요청을 통한 한도 우회 원천 차단 + 비동기 모니터링 누락 제거

P · 1만 건 규모 다운로드를 100건씩 분할 비동기 처리하는 회원제 기능에서, 이용량을 **모든 처리 완료 후** 집계하는 구조 → 사용자가 여러 창에서 동시에 다운로드하면 **검사 시점에 한도가 비어있어 제한 우회**(race condition). 비동기 특성상 실패 다운로드의 모니터링 알림도 누락.

A · 임계구역의 위치 자체를 재설계 — 옵션별로 트레이드오프 비교:

대안 검토 — 동시성 보호 옵션 비교

REJECTED · DB 락

SELECT ... FOR UPDATE

비동기 처리 시간(수 초~수십 초) 동안 락 점유 → **처리량 저하 + 다른 정상 요청까지 블로킹**.

REJECTED · 분산 락

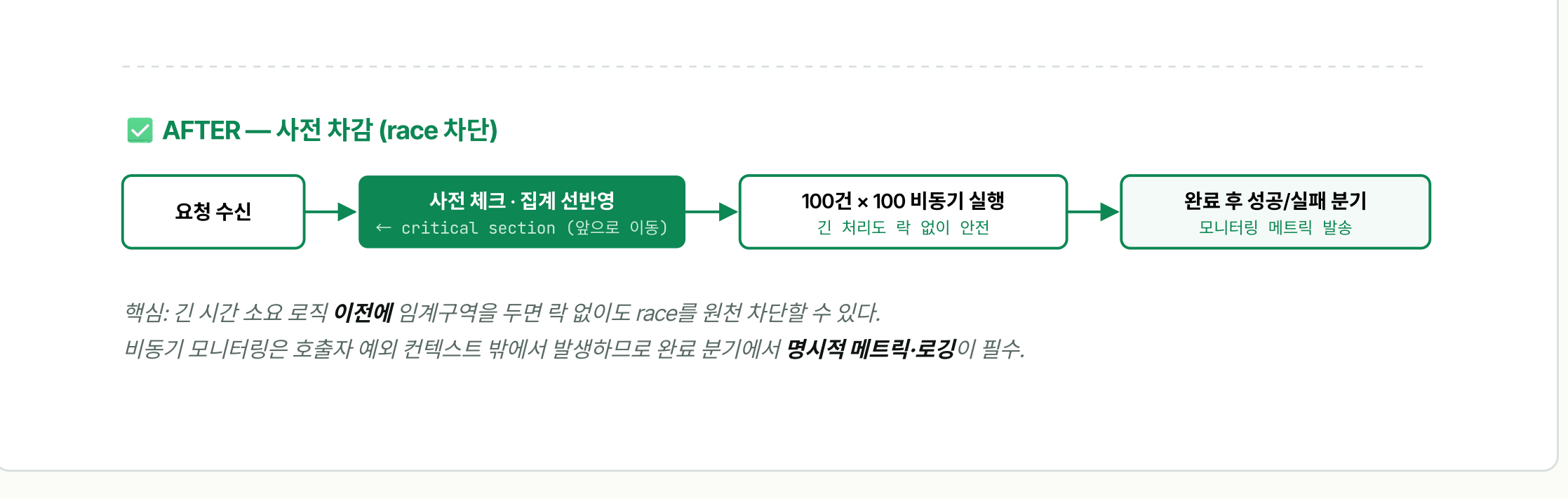
Redis · ZooKeeper

단일 DB 환경에 **오버엔지니어링**. 외부 의존성 추가 + 운영 복잡도 상승. 본 문제는 DB 한 곳에서 풀 수 있음.

CHOSEN · 임계구역 재배치

사전 차감 (Front-loading)

이용량 집계를 **critical section 가장 앞으로 이동**하고 사전 차감 방식 전환. 락 없이도 race 차단 + 실패 시 롤백 가능.



핵심: 긴 시간 소요 로직 ***이전에*** *임계구역을 두면 락 없이도 race를 원천 차단할 수 있다.*
비동기 모니터링은 호출자 예외 컨텍스트 밖에서 발생하므로 완료 분기에서 ***명시적 메트릭·로깅****이 필수.*

R · 동시 다발 요청을 통한 이용량 제한 우회를 **원천 차단**, 비동기 환경 모니터링 누락 제거.

> 당시 한계와 이후 학습
>
> 당시엔 **"여러 방법 중 사전 차감이 맞다"**는 판단으로 해결했으나, 락 기반 대안을 체계적으로 비교하지 못한 상태였음. **이후 학습 결과로 정리한 대안**: 비관적 락(`PESSIMISTIC_WRITE`)으로 차감 row를 잠그거나, 낙관적 락(`@Version`)으로 충돌 시 재시도하는 방식. **해당 환경에서의 판단 근거**: 다운로드 처리 시간이 길어 락 대기 비용이 크므로 사전 차감이 적합. 현재는 환경에 따라 도구를 선택할 수 있는 단계에 도달.

> → 학습 연결 고리
>
> "긴 로직 **이전에** 임계구역을 둔다"는 동시성 설계 원칙을 체득 → 부트캠프에서 **분산 락·동시성 제어를 자청 발표**(낙관·비관·분산 락 트레이드오프) → 다2조부 프로젝트의 **seq 채번 race condition**에서 의식적으로 **비관적 락(PESSIMISTIC_WRITE)** 선택. 3단계 학습 곡선의 시작점.

06 · DEEP DIVE — STREAMING UX

DEEP DIVE — STREAMING UX

대규모 특허 번역 — 서버 일괄 vs 클라이언트 fan-out, 트레이드오프 기반 선택

두 방식의 비용·체감 응답성 트레이드오프를 정리해 의사결정. 인프라 부하 증가를 감수하고 UX를 택한 근거가 분명한 사례

P · 특허 검색 결과 한 화면에 최대 **200건**의 항목이 노출되며, 각 항목의 제목·요약을 사용자의 언어로 번역해 함께 표시해야 함. 번역 API는 건당 수백 ms 단위라 누적 시간이 큼. 팀 내에서 *"번역 도입 후 로딩 속도 저하"*에 대한 우려가 있었음.

A · 두 가지 방식의 트레이드오프를 비교 후, 인프라 수용량과 UX 중요도를 근거로 방식 B 선택. 프론트와 협업해 응답 처리 흐름 설계.

① 방식 A vs B 트레이드오프 정리

A: 서버가 200건 번역 후 한 번에 응답 — 통신은 1회로 효율적이나 사용자는 전부 번역될 때까지 빈 화면 대기. **B**: 원문 즉시 응답 + 클라이언트가 항목별로 번역 fan-out — 서버 통신 1회 → 최대 200회로 증가(**인프라 부하↑**), 대신 첫 화면 대기 시간(TTFB) 극적 단축.

핵심: *"어느 쪽이 빠른가"가 아니라 "어느 쪽 비용을 감당 가능한가 + 어느 쪽 가치가 더 중요한가" 두 축으로 의사결정.*

② B 선택 근거 — 인프라 수용량 + UX 중요도

우리 인프라가 200회 동시 요청 수준을 감당 가능하다는 정보가 있었고, 해당 페이지는 사용자 경험이 중요한 페이지로 분류되어 있었음 → **비용은 감당 가능, 가치는 우위**라는 결론으로 B 채택.

③ 구현 — 원문 즉시 렌더 + 비동기 도착 순 교체

서버는 번역되지 않은 원본을 먼저 빠르게 응답해 화면을 즉시 그림. 이후 클라이언트가 각 항목별로 번역 API를 비동기 호출하고, 응답이 도착하는 순서대로 해당 영역만 원본→번역문으로 교체.

사용자 경험: *빈 화면 0초 → 원본 즉시 노출 → 번역 도착 항목부터 자연스럽게 갱신.*

R · 팀 내에서 우려가 있던 **"번역 도입 후 로딩 속도 저하"가 눈에 띄는 이슈 없이 출시**. 인프라 부하 증가라는 비용을 사전에 인지·감수한 결정이라 출시 후에도 예상 범위 내에서 동작.

> 당시 한계와 이후 학습
>
> 당시엔 트레이드오프를 직접 비교해 적용했으나, 이것이 일반화된 패턴임은 인식하지 못한 상태였음. **이후 학습 결과로 이해한 패턴**: **Client fan-out · Progressive Rendering — TTFB(첫 반응 시간) 단축이 체감 응답성을 결정한다**는 원칙이며, **HTTP 스트리밍·SSE·GraphQL @defer** 등 표준이 같은 철학에서 출발. **다음 작업 시 접근법**: **SSE 같은 표준 프로토콜**이나 **viewport 우선순위 큐**를 함께 검토해 인프라 부하도 같이 줄이는 방향 가능.

07 · DEEP DIVE — VIEWER UX

★ DEEP DIVE — VIEWER UX

이지뷰어 ↔ 도면 팝업 실시간 동기화 — 연간 사용자 만족도 ★ 2위

메인 담당 기능의 핵심 사용성 작업. 단계별로 문제 원인을 짚어 개선한 결과, 그 해 전체 개선 기능 중 사용자 만족도 2위로 선정

P · 메인 창에는 다수의 특허 문헌을 빠르게 탐색할 수 있는 **"이지뷰어"**가 있고, **"도면 보기"**를 클릭하면 별도 컨트롤러 URL을 호출해 전체 HTML을 받아오는 **팝업 창**이 열리는 구조. *A문헌 도면을 본 상태에서 이지뷰어로 다른 문헌을 탐색해도 팝업은 A문헌 도면 상태로 고정*되어, 사용자가 매번 "도면 보기"를 다시 클릭해야 하는 불편이 있었음.

A · 이지뷰어 문헌 이동 시 팝업 도면이 자동 동기화되도록 개선. **각 단계에서 드러난 문제 원인을 정확히 짚어 다음 단계로 이어감**.

1차 — 팝업 자동 갱신 (URL 재호출)

B문헌으로 이동 시 도면 조회 컨트롤러 URL을 다시 호출해 **팝업 창 전체를 새로 불러오는 방식**.

남은 문제 3가지: *① 페이지 전체 재로딩으로 느림 ② 화면이 순간 하얗게 깜빡임 ③ 새 URL 로드 시* ***브라우저 보안 정책에 의해 포커스가 팝업 창으로 강제 이동*** *→ 이지뷰어에서 빠르게 문헌을 넘기던 탐색 흐름이 매번 끊김.*

2차 — 부분 렌더링 (동기 AJAX)

깜빡임·포커스 강탈의 근본 원인이 **"팝업 URL을 바꿔 페이지 전체를 재로딩"하는 데 있다고 판단**. 팝업 HTML을 **고정 영역(헤더·툴바) + 가변 영역(도면)**으로 분할하고, 팝업 URL은 그대로 둔 채 **가변 영역의 HTML 조각만 동기 AJAX로 받아 innerHTML로 교체**.

해결: *깜빡임·포커스 강탈 사라지고 전송량 감소로 속도 개선.* ***남은 문제:*** *동기 AJAX 특성상 응답 도착 전까지 메인 스레드가 멈춰 "순간 멈춤"이 인지됨.*

3차 — 비동기 호출 + 콜백 기반 실행 순서 재구성

AJAX를 비동기로 변경. 단순히 동기→비동기 전환만으로는 부족하고, **응답 시점에 의존하는 후속 로직(도면 초기화 JS 등)이 응답 완료 후 실행되도록 콜백 기반으로 코드 구조를 재정비**.

해결: *응답 대기 중에도 사용자 입력에 반응 가능해져 멈춤 현상까지 해소. 1·2차에서 드러난 모든 문제(깜빡임·포커스 강탈·속도·멈춤)가 최종적으로 모두 개선됨.*

R · **연간 사용자 만족도 조사에서 그 해 개선 기능 중 2위로 선정**. *"기능이 동작하는 것"에 멈추지 않고 "사용자 흐름을 끊지 않는 것"까지 가져간 점*이 실질적 평가 근거.

> 회고
>
> 단순히 *"AJAX를 썼다"*가 아니라, 각 단계의 문제 원인을 정확히 짚어 다음 단계로 이어간 점이 핵심. **페이지 전체 재로드 → 부분 교체 → 비동기화**로의 흐름은 결과적으로 **"브라우저가 강제하는 동작을 피하기 위해 점점 더 가벼운 단위로 갱신 범위를 좁혀 나간 과정"**이었음.

> → 작업의 의미
>
> 단순 *기능 추가*가 아니라 **"브라우저가 강제하는 동작 단위를 좁혀 사용자 흐름을 보존하는"** 관점의 설계. 사용자 경험을 코드 구조 레벨에서 직접 풀어낸 작업.
>
> **점진적 응답성 개선(전체 재로딩 → 부분 렌더링 → 비동기 갱신)**의 사고 패턴은 이후 **대규모 특허 번역**(서버 일괄 응답 → 클라이언트 fan-out + 점진 교체)으로 이어지고, 부트캠프 프로젝트의 *ants-camp 클라이언트 fan-out 구조*까지 일관되게 적용됨.

08 · STACK

경력 중 사용 스택

영역	스택
Language / Framework	**Java 8 · Spring 4.x (Legacy)** · JavaScript
Data Access	**MyBatis** · Stored Procedure (T-SQL)
Database	PostgreSQL → **MSSQL 마이그레이션**
Document Parsing	**Apache POI** (MS Office) · **hwplib** (HWP) — E-DRM 시그널 식별
Test	JUnit · Mockito
Frontend (Legacy)	JSP · **AngularJS** · jQuery 계열 — 비동기 렌더링·형광펜 레이어·스크롤·이미지 동기화

위는 **레거시 코드베이스 운영 환경**의 스택. 부트캠프 프로젝트(Java 17 · Spring Boot 3.5 · React/TS 등)에서 **모던 스택으로 학습·적용 병행** → 다음 섹션(GROWTH MAP) 참조.

09 · GROWTH MAP

실무 한계 → 학습 동기 → 부트캠프 적용

실무에서 시행착오로 풀어낸 문제들이 그대로 **"왜 그게 효과 있었는지 알고 싶다"**는 학습 동기로 이어졌습니다. 이론으로 채운 뒤 다시 프로젝트에서 검증하는 사이클이 두 번 완결.

GROWTH CYCLE #1 — SQL · 벤더 종속성 · DB 깊이

PG → MSSQL 슬로우 쿼리 시행착오 → ORM 학습 + 실행 계획 공부 → 부트캠프 프로젝트 적용

실무 · 100% MyBatis 환경에서 슬로우 쿼리를 **경험적 시행착오로 풀었지만**, "왜 효과 있는지" 설명할 수 없는 부분이 마음에 남음.

학습 · 영속성 컨텍스트·JPQL·벤더 독립 데이터 접근 계층 설계를 학습. 옵티마이저·실행 계획·카디널리티 추정 개념도 함께 정리. **"왜 ORM이 필요한지"를 실무 경험으로 설명 가능**.

적용 · 부트캠프 프로젝트에서 **JPA + QueryDSL**로 데이터 액세스 계층 직접 설계. PostGIS 같은 벤더 특수 기능은 Native Query로, 일반 도메인은 JPA로 — 이 분기 기준도 실무에서 체감한 한계로부터 나온 판단.

GROWTH CYCLE #2 — 동시성 제어

비동기 다운로드 race 직감으로 해결 → 락 트레이드오프 발표 → 다2조부 비관적 락 의식적 선택

실무 · "긴 로직 이전에 임계구역을 둔다"는 방식으로 race를 풀었지만, **어떤 락이 어떤 상황에 맞는지 체계적으로 모름**.

학습 · 부트캠프 스터디에서 **낙관적/비관적/분산 락 트레이드오프**를 자청 발표. 각 락의 동작 원리·임계구역 크기·충돌 빈도에 따른 선택 기준을 코드 레벨로 검증.

적용 · 다2조부 프로젝트의 **seq 채번 race condition**에 비관적 락(`PESSIMISTIC_WRITE`) 적용. 분산 락도 검토 후 "단일 인스턴스 토폴로지엔 오버엔지니어링"이라는 판단으로 배제 — **여러 옵션을 알고 의도적으로 선택**한 사례.

← 포트폴리오 메인으로

danbeekimm/da2joburureung

01 · OVERVIEW

정합성 보장과 장애 격리를 설계한 B2B 물류 MSA

직전 모놀리식(omin)에서 한 도메인 결함이 전체를 멈추고, 특정 도메인 전용 기술(PostGIS)을 전 팀원이 설치해야 했던 비효율을 직접 겪고 **도메인별 독립 배포·기술 선택이 가능한 MSA**로 확장. B2B 도메인을 코드에 녹이기 위해 **DDD 4계층(헥사고날)**을 적용하고, 서비스 간 통신은 "**검증=동기(Feign), 통지=비동기(Kafka)**"로 의도적으로 분리했습니다.

> "분산 트랜잭션 없이 정합성을, 장애 전파 없이 통신을."
>
> 전국 17개 허브를 거점으로 한 **스파르타 물류** — 업체·허브·상품·주문·배송을 MSA로 분리한 B2B 물류 시스템에서 **company-service 전담 + DeliveryManager 도메인** 설계. Outbox 패턴 · Circuit Breaker · OR-Tools VRPTW · 비관적 락을 비즈니스 근거에서 출발해 채택.
>
> ● 2026 · 부트캠프 5인 ● 담당: company-service + delivery-service 내 3개 도메인
>
> ● Spring Cloud 2025 · Kafka · OR-Tools · Keycloak

at-least-once
이벤트 정합성 보장
Outbox 패턴 · 분산 트랜잭션 없이

5분 → 3초
배송 배정 자동화
수동 배정 → OR-Tools VRPTW (100건 기준 · 데모)

DB-level
race 원천 차단
비관적 락 · 재시도 로직 없이

02 · SERVICE OVERVIEW

스파르타 물류 — MSA 기반 국내 B2B 물류 관리 및 배송 시스템

전국 17개 지역 허브 센터를 거점으로 한, "공급업체 → 허브 → 허브 → 수령업체" 전 구간을 자동화한 B2B 물류 관리 플랫폼.

스파르타 물류는 전국 17개 도/광역시에 허브 센터를 보유한 가상의 B2B 물류 회사입니다. 각 허브는 여러 업체의 물건을 보관하며, 주문이 발생하면 시스템은 **출발 허브 → 목적지 허브 간 화물 이동(허브 배송 담당자)**과 **최종 허브 → 수령 업체 배송(업체 배송 담당자)** 두 트랙을 분리해 처리합니다.

예시 시나리오 — "부산 수산물 도매 업체가 경기도 일산의 건조 식품 가공품 50개를 주문" → 시스템이 **경기도 허브 → 부산 허브**로 화물 이동을 계획하고, 도착하면 **부산 허브 소속 업체 배송 담당자**가 최종 업체로 전달.

시스템 핵심 도메인 (8개)

01 · COMPANY ★ 담당
업체 관리 (생산 · 수령)
생산업체·수령업체로 구분, 모든 업체는 특정 허브에 소속. 업체 삭제 시 주문/상품/배송 등 타 서비스에 이벤트 통지 필요.

02 · HUB · HUBPATH
17개 허브 · 허브 간 경로
전국 도/광역시 17개 센터 관리 + 허브 간 이동 경로 매핑. 모든 화물의 시작/도착점.

03 · PRODUCT
상품 · 재고 관리
모든 상품은 특정 업체와 허브에 소속. 주문 발생 시 재고 감소, 취소 시 복원.

04 · ORDER
주문 라이프사이클
주문 생성 → 재고 검증·차감 → 배송 트리거. 재고 부족 시 주문 실패. 삭제 시 논리 삭제(deleted_at).

05 · DELIVERY ★ 담당
배송 · 담당자 · 경로 (3개 도메인 통합)
배송 본체 + DeliveryManager(허브당 10명 + 전체 10명) + DeliveryRoute(경로 기록)를 한 컨텍스트로 통합. **매일 06:00 VRPTW 자동 배정.**

06 · USER · AUTH
4단계 권한 관리
마스터 / 허브 관리자 / 배송 담당자 / 업체 담당자. **Keycloak + JWT**로 권한 검증.

07 · NOTIFICATION
Slack 메시지 관리
시스템·사용자 발신 슬랙 메시지 전부 엔티티에 저장. Incoming Webhook으로 외부 발송.

08 · AI
AI 상품 정보 생성
상품 등록 시 LLM 기반 설명 자동 생성. AI 응답 로그는 별도 트랜잭션(REQUIRES_NEW)으로 분리해 본 로직 무영향.

2-Track 배송 구조 — 발제문 기반

트랙	역할	인원	배정 방식
허브 배송 담당자	허브 간 화물 이동 (경기도 허브 → 부산 허브)	전체 10명	대기시간·순번 기반 **라운드로빈**
업체 배송 담당자	최종 허브 → 수령 업체 배송	허브당 10명	**OR-Tools VRPTW** (시간창·차량 부하 균등)

두 트랙의 **업무는 명확히 분리** — 허브 배송 담당자는 허브 → 업체 배송 불가, 업체 배송 담당자는 허브 간 배송 불가. 이 도메인 특성을 코드로 강제하기 위해 **배정 알고리즘 자체를 트랙별로 분리**(라운드로빈 / VRPTW)했습니다.

03 · DESIGN GOALS

설계 도전 과제와 핵심 기여

핵심 도전 과제

- 분산 트랜잭션 없이 **이벤트 유실 · 유령 이벤트** 차단 — DB 커밋과 Kafka 발행의 원자성 불가
- 외부 서비스 장애 시 **연쇄 전파(cascading failure)** 차단 — 대기 스레드 누적 문제
- B2B의 **시간창 SLA** 준수 — 후처리 정렬 아닌 제약 조건으로 모델링 필요
- seq 채번의 **Read-Modify-Write race** — 동시 등록 시 중복 발급 위험

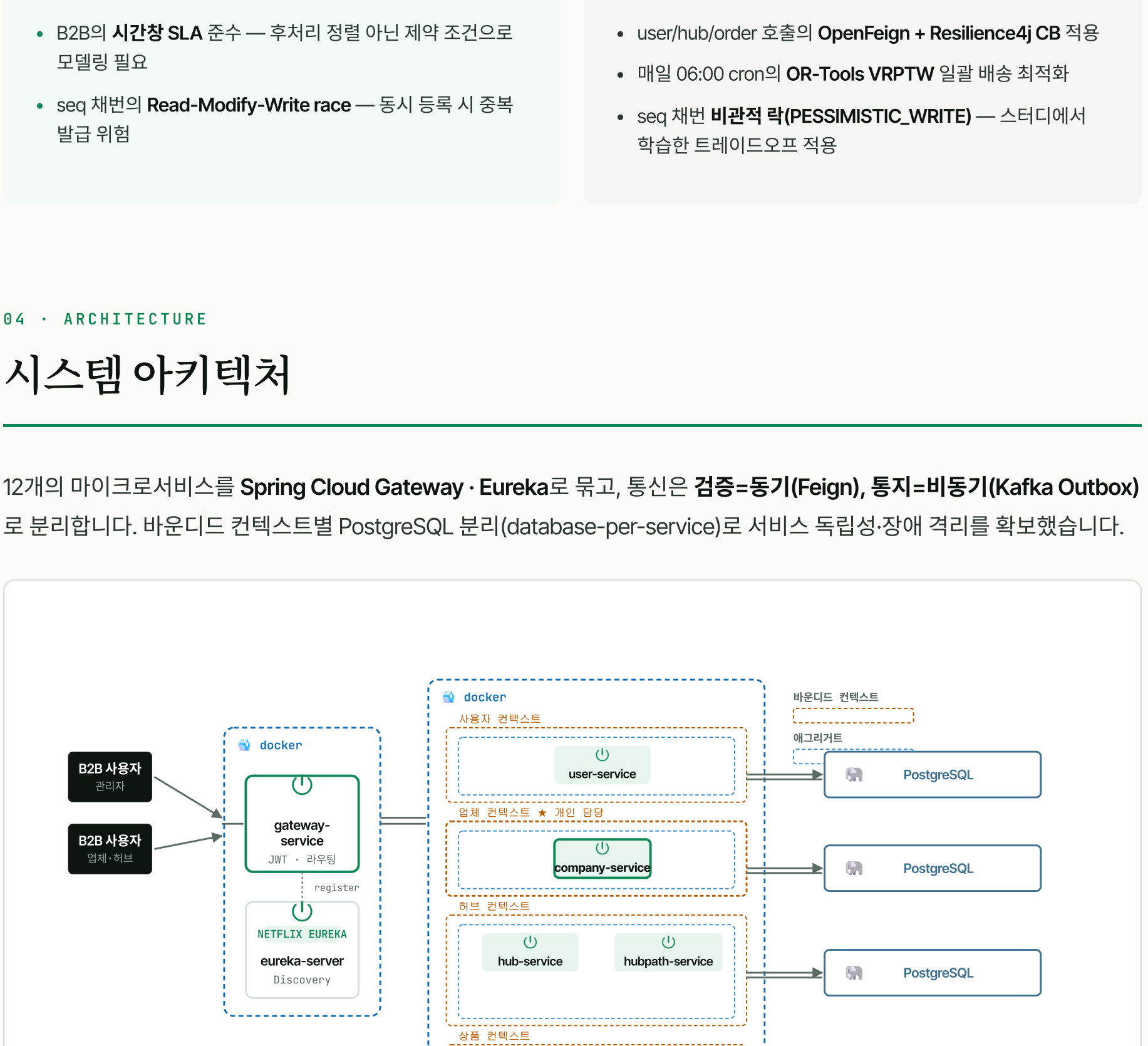
핵심 기여 (개인)

- company-service 전담 + DeliveryManager 도메인 **DDD 4계층 헥사고날** 설계
- 업체 삭제 이벤트의 **Kafka + Outbox 패턴** 구현 (Saga 대안 검토 후 채택)
- user/hub/order 호출의 **OpenFeign + Resilience4j CB** 적용
- 매일 06:00 cron의 **OR-Tools VRPTW** 일괄 배송 최적화
- seq 채번 **비관적 락(PESSIMISTIC_WRITE)** — 스터디에서 학습한 트레이드오프 적용

04 · ARCHITECTURE

시스템 아키텍처

12개의 마이크로서비스를 **Spring Cloud Gateway · Eureka**로 묶고, 통신은 **검증=동기(Feign), 통지=비동기(Kafka Outbox)**로 분리합니다. 바운디드 컨텍스트별 PostgreSQL 분리(database-per-service)로 서비스 독립성·장애 격리를 확보했습니다.



*설계 의도: 12개 서비스를 7개 바운디드 컨텍스트(파선)로 묶고, 각 컨텍스트는 자기 PostgreSQL을 가짐(database-per-service). 배송 컨텍스트는 처음엔 `delivery / delivery_manager / delivery_route` 3개 서비스로 분할 시도했으나, **강한 응집성·동일 트랜잭션 경계 요구** 때문에 `delivery-service` 한 컨텍스트로 통합하고 내부 도메인으로 분리.*

설계 의사결정과 근거

결정	대안	선택 이유 — 트레이드오프 명시
MSA + DDD 헥사고날	모놀리식 (omin)	직전 omin에서 한 도메인 결함이 전체를 멈추고 도메인 전용 기술(PostGIS)을 전 팀원이 설치해야 했던 비효율 직접 경험 → **도메인별 독립 배포·기술 선택**이 가능한 MSA로.
database-per-service	공유 DB · schema 분리	바운디드 컨텍스트별 PostgreSQL 분리 → 서비스 독립성·장애 격리. **데이터 정합성은 Outbox 이벤트로 별도 보장.**
"검증=동기, 통지=비동기" 통신 분리	전부 동기 / 전부 비동기	정합성 검증("이 hub가 있나?")은 응답을 받아야 진행 가능 → 동기(Feign). 삭제 통지는 응답 불필요 → 비동기(Kafka). **의도적 분리.**
Outbox 패턴	Saga · 2PC · 직접 Kafka 발행	단방향 통지·응답 불필요라 full Saga는 과함. 2PC는 분산 트랜잭션 회피. 직접 발행은 유실/유령 이벤트 위험 → **at-least-once + 결과적 일관성**의 Outbox가 적정.
단일 인스턴스 → 분산 락 생략	ShedLock · 리더 선출	각 서비스 단일 인스턴스 배포 토폴로지 → Outbox 스케줄러 중복 실행 자체가 발생 X. **수평 확장 시** ShedLock + 컨슈머 멱등 처리 추가 예정.

05 · KEY CONTRIBUTION ①

CONTRIBUTION #1

이벤트 유실·유령 이벤트 없는 비동기 통지 — Kafka + Outbox 패턴

분산 트랜잭션 없이 at-least-once · 결과적 일관성 보장. 단위 테스트로 단일 이벤트 실패가 나머지 발행을 막지 않음 검증

P · 업체 삭제 시 주문·상품 등 타 서비스 통지가 필요한데, "DB 커밋 → Kafka 발행" 사이 발행 실패 시 **이벤트 영구 유실**(DB는 지워졌는데 남들은 모름), 발행 후 DB 롤백 시 **유령 이벤트**(다른 서비스는 삭제됐다고 알고 처리). 분산 트랜잭션(2PC)은 가용성을 해치고 MSA와 부적합.

A · 삭제 트랜잭션 내 `OutboxEvent` (PENDING) **원자 저장** → DB 커밋과 함께 커밋(또는 함께 롤백). `OutboxEventScheduler` 가 `@Scheduled(fixedDelay=10000)` 로 10초마다 PENDING을 Kafka로 발행→ `PUBLISHED` . 실패 시 PENDING 유지 → 다음 주기 자동 재발행.

R · 분산 트랜잭션 없이 **at-least-once 발행 + 결과적 일관성** 보장. 단위 테스트로 단일 이벤트 실패가 나머지 발행을 막지 않음 검증.

CODE CompanyService.java · deleteCompany 원자 저장 | OutboxEventScheduler.java · 10s 폴링 발행 | OutboxEvent.java · 상태 모델 | OutboxEventSchedulerTest.java · 단위 테스트

대안 검토 — 옵션별 트레이드오프

REJECTED · 2PC
분산 트랜잭션 코디네이터
모든 참여자가 잠금을 유지 → **가용성 저하**, MSA와 부적합. 코디네이터 자체가 SPOF.

REJECTED · Saga
보상 트랜잭션 체인
다중 단계 트랜잭션엔 강력하나, 본 케이스는 **단방향 통지·응답 불필요**라 full Saga는 과함. 보상 로직 작성 비용도 큼.

CHOSEN · OUTBOX
트랜잭셔널 Outbox
도메인 변경과 이벤트를 **같은 트랜잭션에 기록** → 별도 publisher가 polling으로 Kafka 발행 → 실패 시 PENDING 유지·재시도. **at-least-once**.

```java
// 업체 삭제 트랜잭션 내 Outbox 원자 저장
@Transactional
public void deleteCompany(UUID companyId) {
    companyRepository.delete(companyId);
    outboxRepository.save(new OutboxEvent(
        "company.deleted", companyId, payload, "PENDING"
    ));  // 같은 트랜잭션 — 원자 보장
}

// 10초마다 PENDING → Kafka
@Scheduled(fixedDelay = 10000)
public void publish() {
    outboxRepository.findByStatus("PENDING").forEach(event -> {
        kafkaTemplate.send(event).addCallback(
            ok -> outboxRepository.markPublished(event.getId()),
            fail -> log.warn("retry next tick")
        );
    });
}
```

06 · KEY CONTRIBUTION ②

CONTRIBUTION #2

검증=동기, 통지=비동기 — Feign + Circuit Breaker로 통신 의도 분리

외부 서비스 장애의 연쇄 전파 차단 — CB(window 10·실패율 50%·OPEN 10s·slow-call 3s) + Retry 3회로 fallback 즉시 반환

P · 외부 서비스 호출이 타임아웃까지 블로킹되면 **대기 스레드 누적 → 내 서비스까지 연쇄로 죽음**(cascading failure). 단순 동기 호출은 한 서비스 장애가 전체로 번지는 구조.

A · 통신 의도를 의도적으로 분리: **"정합성 검증=동기(Feign)", "삭제 통지=비동기(Kafka)"**. user/hub/order 호출은 `@CircuitBreaker` + `@Retry` 로 감쌈. CB(window 10·실패율 50%·OPEN 10s·slow-call 3s·retry 3회/500ms)로 fallback 즉시 반환.

R · CB OPEN 시 즉시 fallback 반환 → 대기 스레드 누적·연쇄 장애 차단. CB 상태전이(OPEN/CLOSED/HALF_OPEN)·호출차단·slow-call 전체 로깅.

CODE HubClientImpl.java · CB+fallback | ResilienceConfig.java · 상태전이 로깅 | application-docker.yml · CB 설정값

```java
@Retry(name = "hubService")
@CircuitBreaker(name = "hubService", fallbackMethod = "hubServiceFallback")
public boolean validateHubExists(UUID hubId) {
    return hubClient.exists(hubId);  // 동기 검증
}

private boolean hubServiceFallback(UUID hubId, Throwable t) {
    log.warn("Hub validation fallback — CB OPEN. hubId={}", hubId);
    return false;  // 즉시 반환 → 스레드 점유 X
}
```

설정 항목	값	이 값으로 정한 이유
sliding window	10 calls	실패율 산출 표본 크기. 너무 작으면 1~2건 실패에 과민 반응, 너무 크면 장애 감지가 늦음 → 둘의 절충점
failure rate threshold	50%	retry 3회를 통과하고도 절반이 깨지면 일시적 결함이 아니라 명백한 장애로 판단. 그 이하(예: 30%)면 단발성 오류에도 차단돼 과민
wait in OPEN	10s	외부 서비스 재기동·순간 부하 해소에 통상 수~십수 초. 더 짧으면 복구 전 재요청이 몰려 재차 OPEN, 더 길면 정상화 후에도 차단 지속
half-open calls	3	1건 성공은 우연일 수 있어 성급한 CLOSE를 방지. 3건 시험 호출로 실제 복구 여부를 확인 후 닫음
slow call threshold	3s	등록 플로우가 허용할 수 있는 지연 상한. Feign read timeout보다 짧게 둬 '느린 성공'도 장애로 간주해 미리 차단
retry	3회 / 500ms	패킷 손실·순간 GC 같은 일시 결함은 보통 1~2회 재시도로 흡수. 500ms 간격으로 순간 부하 분산, 3회 초과 시 일시 결함이 아니라 보고 CB에 위임

값 선정 전제 — 데모 트래픽 기준의 보수적 baseline. Resilience4j 기본값을 출발점으로 위 근거에 따라 조정했고, **운영 전환 시 실제 호출량·지연 분포로 부하 테스트 후 재튜닝**하는 것을 전제로 한 초기값.

07 · KEY CONTRIBUTION ③

CONTRIBUTION #3

시간창을 제약으로 모델링한 일괄 배송 최적화 — OR-Tools VRPTW

B2B SLA의 핵심(시간창 준수)을 사후 정렬이 아닌 솔버의 직접 제약으로 모델링. 수동 5분 → 자동 3초 (100건 기준 · 데모)

P · B2B 물류는 업체 영업시간(예: "오전 9~11시 입고")이 **SLA의 핵심**. 어기면 계약 위반 → **시간창을 제약(constraint — 솔버가 반드시 지켜야 하는 조건)으로 모델링**해야지, 후처리 정렬로는 보장 불가. 단순 라운드로빈(들어온 순서대로 담당자에게 균등 배분)은 거리·시간창·차량 부하 균등을 못 고려해 비효율.

A · 매일 06:00 스케줄러가 허브별 당일 배송을 모아 **VRPTW로 일괄 최적화**. 트랙 분리 — **허브 간 = 라운드로빈, 업체 = OR-Tools VRPTW**. 제약을 모두 만족하는 해가 없으면(infeasible) 시간창을 단계적으로 완화해 재시도.

> **입력** — Haversine 공식(위·경도 좌표로 두 지점 간 직선거리 계산)으로 시간·거리 행렬 생성
>
> **제약** — 시간창 ±30분 · 라우트 최대 480분(8시간) · 차량당 상한 `ceil(총건/담당자수)` 로 부하 균등
>
> **솔버** — 가까운 지점부터 이어 붙여 초기 경로를 빠르게 만드는 휴리스틱(`PATH_CHEAPEST_ARC`) + 그 경로를 반복 개선하며 부분 최적에 갇히지 않도록 탐색하는 기법(`GUIDED_LOCAL_SEARCH`), 탐색 5초 제한
>
> **가용성** — 해가 없을 때(infeasible) 시간창 단계적 완화 + 스케줄러 레벨 3회 재시도(10초 간격)

R · 시간창을 솔버의 직접 제약으로 모델링해 **SLA 보장 가능한 자동 배정 체계** 확보. 해가 없을 때도 시간창 완화로 가용성 확보.

CODE OrToolsRouteOptimizationService.java · 솔버 구성 | CompanyDeliveryAssignmentService.java · 시간창 완화 | CompanyDeliveryAssignmentScheduler.java · 06:00 cron+재시도 | HubDeliveryAssignmentService.java · 라운드로빈

대안 검토 — 왜 OR-Tools인가

REJECTED · K-means
거리 기반 군집화
시간창·차량용량을 **제약으로 못 넣음**. 클러스터링 후 시간창 검증은 사후 필터링 → 위반 건은 다음 날로 밀려 **SLA 위반**. 초기값에 의존하는 휴리스틱(경험적 어림법)이라 결과도 불안정.

REJECTED · pgRouting
경로 탐색 (A→B 최단)
경로 탐색엔 강하지만 **"어떤 묶음을 누구에게 배정"하는 할당(assignment) 문제는 안 풂**. 지리 공간 시각화·근사 계산용에 가깝고, 제약 만족 문제(CSP — 여러 제약을 동시에 만족하는 해를 찾는 문제)에는 부적합.

CHOSEN · OR-TOOLS
제약 만족 솔버 (CSP)
시간창·차량 용량·작업 시간을 **솔버가 직접 지키는 제약으로 선언** → 솔버가 제약 안에서 최적해 탐색. 해가 없으면 시간창 완화 단계적 재시도 가능.

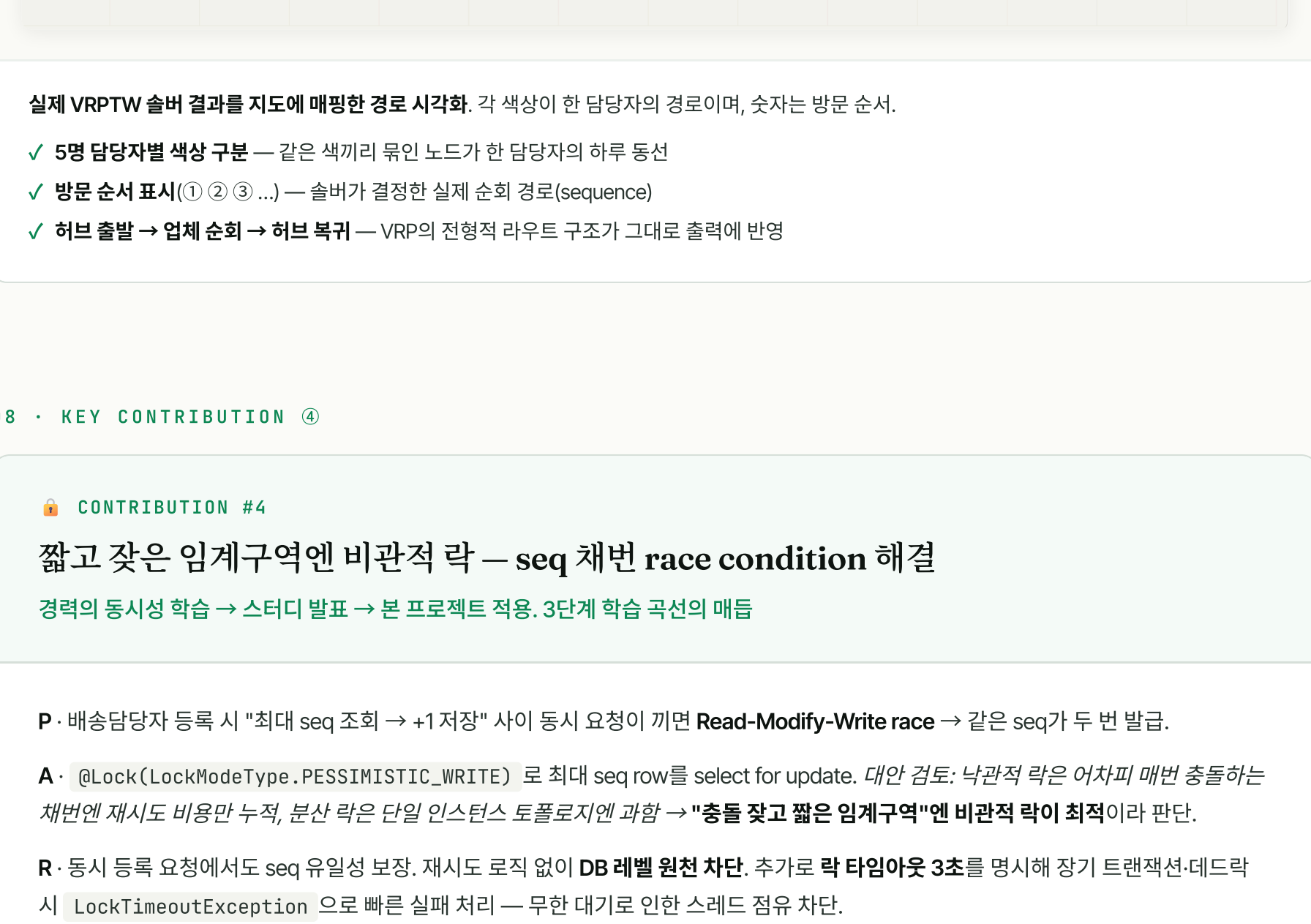
OR-Tools VRPTW 구성요소 한눈에

개념	코드 표현	의미
Depot	`depot` 인덱스	모든 라우트의 출발·복귀 지점(허브)
Dimension	`AddDimension(...)`	라우트 따라 누적되는 값(시간·거리·용량)에 제약 부여
Time Window	Dimension slack + 노드별 setRange	업체별 영업시간 — 도착 가능 구간
1st Solution Strategy	`PATH_CHEAPEST_ARC`	초기해를 빠르게 만드는 휴리스틱 (가까운 노드 우선)
Local Search Metaheuristic	`GUIDED_LOCAL_SEARCH`	초기해를 점진적으로 개선 — 국소최적 탈출 메커니즘
Time Limit	`setTimeLimit`	솔버 탐색 시간 상한 — 충분히 좋은 해에서 stop

```java
// VRPTW 핵심 구성
// nodes: 방문 지점 수(허브+업체) / vehicles: 담당자 수 / depot: 출발·복귀 지점(허브) 인덱스
RoutingIndexManager manager = new RoutingIndexManager(
    nodes, vehicles, depot
);

// AddDimension: 시간/거리/용량처럼 라우트 따라 누적되는 값에 제약 부여
routing.AddDimension(
    timeCallback,
    30,        // 노드별 대기 가능 여유(slack) — 시간창 도착에 30분 버퍼
    480,       // 한 담당자 라우트 총 시간 상한(분) = 8시간
    false, "Time"  // depot 시작 시점에 누적값 리셋 안 함
);

// 차량당 상한 — 부하 균등
int maxPerVehicle = (int) Math.ceil((double) total / vehicles);

// 솔버 구성 = "초기해를 빠르게 + 그 해를 개선" 2단
RoutingSearchParameters params = main.defaultRoutingSearchParameters()
    .toBuilder()
    .setFirstSolutionStrategy(FirstSolutionStrategy.PATH_CHEAPEST_ARC)         // 가까운 노드부터 그리디로 초
    .setLocalSearchMetaheuristic(LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH) // 초기해를 개선하며 국소최적
    .setTimeLimit(Duration.ofSeconds(5))
    .build();
```

왜 일괄(batch)인가 — 실시간 배정 = 들어오는 순서대로 그리디 → **국소 최적**(지금 가까운 사람에게 줬더니 나중 배송이 꼬임). VRPTW는 **당일 전체를 한 번에 모아야 전역 최적**. B2B는 "즉시"보다 "당일 정시 배송"이 중요 → 일괄이 비즈니스에 더 맞음.

실제 OR-Tools VRPTW 솔버 출력 — 담당자별 경로 시각화 VRPTW · 실측



실제 VRPTW 솔버 결과를 지도에 매핑한 경로 시각화. 각 색상이 한 담당자의 경로이며, 숫자는 방문 순서.

- ✓ **5명 담당자별 색상 구분** — 같은 색끼리 묶인 노드가 한 담당자의 하루 동선
- ✓ **방문 순서 표시**(① ② ③ ...) — 솔버가 결정한 실제 순회 경로(sequence)
- ✓ **허브 출발 → 업체 순회 → 허브 복귀** — VRP의 전형적 라우트 구조가 그대로 출력에 반영

08 · KEY CONTRIBUTION ④

CONTRIBUTION #4

짧고 잦은 임계구역엔 비관적 락 — seq 채번 race condition 해결

경력의 동시성 학습 → 스터디 발표 → 본 프로젝트 적용. 3단계 학습 곡선의 매듭

P · 배송담당자 등록 시 "최대 seq 조회 → +1 저장" 사이 동시 요청이 끼면 **Read-Modify-Write race** → 같은 seq가 두 번 발급.

A · `@Lock(LockModeType.PESSIMISTIC_WRITE)` 로 최대 seq row를 select for update. *대안 검토: 낙관적 락은 어차피 매번 충돌하는 채번엔 재시도 비용만 누적, 분산 락은 단일 인스턴스 토폴로지엔 과함* → **"충돌 잦고 짧은 임계구역"엔 비관적 락이 최적**이라 판단.

R · 동시 등록 요청에서도 seq 유일성 보장. 재시도 로직 없이 **DB 레벨 원천 차단**. 추가로 **락 타임아웃 3초**를 명시해 장기 트랜잭션·데드락 시 `LockTimeoutException` 으로 빠른 실패 처리 — 무한 대기로 인한 스레드 점유 차단.

CODE JpaDeliveryManagerRepository.java · @Lock+timeout 3000

REJECTED · 낙관적 락
버전 충돌 → 재시도
채번은 **매번 같은 row를 만짐** → 거의 항상 버전 충돌 발생 → 재시도 비용만 누적되고 처리량 저하.

REJECTED · 분산 락
Redis · ZooKeeper
단일 인스턴스 배포 토폴로지엔 **오버엔지니어링**. 외부 의존성도 늘어남.

CHOSEN · 비관적 락
SELECT ... FOR UPDATE
임계구역이 **짧고 충돌이 잦음** → 락 대기 단점 최소화, 충돌 차단 장점 극대화. 재시도 로직 없이 DB 레벨 원천 차단.

```java
@Lock(LockModeType.PESSIMISTIC_WRITE)
@QueryHints({@QueryHint(name = "jakarta.persistence.lock.timeout", value = "3000")})
@Query("select dm from DeliveryManager dm "
     + "where dm.hubId = :hubId order by dm.seq desc")
Optional<DeliveryManager> findTopByHubIdOrderBySeqDesc(UUID hubId);
// → SELECT ... FOR UPDATE — 동시 요청은 대기 후 순차 처리, 3초 초과 시 LockTimeoutException
```

3단계 학습 곡선 연결 — 전 회사의 **비동기 다운로드 race condition**을 해결하면서 "정확히 어떤 락이 어느 상황에 맞는지" 몰랐던 게 학습 동기. 부트캠프 스터디에서 **낙관적/비관적/분산 락 트레이드오프**를 자청 발표. 본 프로젝트에서 **"충돌 잦고 짧은 임계구역엔 비관적 락"**을 실제로 적용 — 한 사이클 완결.

09 · TROUBLESHOOTING

트러블슈팅 — 가설을 세우고 대안을 비교한 사례

#01 · VRPTW infeasible — 시간창이 빡빡해 해 없음

제약 완화 재시도

현상 · 가설
- →특정 일자에 업체 시간창 좁고 배송 물량 집중
- →OR-Tools가 **infeasible** 반환 → 자동 배정 실패
- →단순 예외 throw는 가용성 저하 → **제약 완화 전략 필요**

원인 분석 — 대안 검토
- →차량 추가 투입: 인력 자원 문제 — 시스템 제어 밖
- →모든 제약 무시 후 재계산: SLA 약속이 깨짐
- →**시간창만 단계적 완화**: SLA 영향 최소화 + 가용성 확보

해결 & 검증
- →시간창 **±30분 → ±60분 → ±90분** 단계적 완화 재시도
- →모두 실패 시 `ROUTE_OPTIMIZATION_FAILED` 예외로 운영자 알림
- →스케줄러 레벨 **3회 수동 재시도**(10초 간격)

CODE CompanyDeliveryAssignmentService.java | CompanyDeliveryAssignmentScheduler.java

#02 · 외부 서비스 장애가 연쇄 전파 — Circuit Breaker로 격리

cascading failure

현상 · 가설
- →hub-service 일시 장애 시 company-service의 검증 호출이 타임아웃까지 블로킹
- →**대기 스레드 누적** → company-service 자체 응답 지연

원인 분석 (WHY → WHY)
- →외부 호출 타임아웃까지 스레드 점유
- →풀에서 스레드가 빨리 소진
- →새 요청이 대기 큐로 쌓이고 결국 자기도 타임아웃
- →fail-fast가 없어 장애가 양방향 전파

해결 & 검증
- →Resilience4j CB로 감싸 **OPEN 시 fallback 즉시 반환**
- →스레드 점유 시간 = 타임아웃 → 수 ms로 단축
- → `onStateTransition` · `onCallNotPermitted` · `onSlowCallRateExceeded` 로깅 검증

CODE HubClientImpl.java | ResilienceConfig.java

#03 · seq 채번 동시성 — 비관적 락으로 원천 차단

race condition

현상 · 가설
- →배송담당자 동시 등록 시 **Read-Modify-Write race**
- →같은 seq가 두 번 발급될 수 있음

원인 분석 — 대안 비교
- →낙관적 락: 매번 충돌하는 채번엔 재시도 비용만 누적
- →분산 락: 단일 인스턴스 토폴로지엔 과함
- →**비관적 락**: 짧고 잦은 임계구역에 최적

해결 & 검증
- → `@Lock(PESSIMISTIC_WRITE)` 로 최대 seq row를 `SELECT FOR UPDATE`
- →재시도 로직 없이 DB 레벨 원천 차단
- →동시 등록 시나리오 단위 테스트로 유일성 검증

CODE JpaDeliveryManagerRepository.java

10 · STACK

기술 스택

영역	스택
Backend	Java 17 · Spring Boot 3.5.12 · **Spring Cloud 2025.0.1** (Eureka · Gateway · OpenFeign)
Messaging	Spring Kafka · **Outbox 패턴**
Resilience	Resilience4j · **Circuit Breaker · Retry**
Data	PostgreSQL (database-per-service) · Redis · JPA · QueryDSL
Optimization	**Google OR-Tools 9.12.4544 (VRPTW)**
Infra	Docker Compose

11 · RETROSPECTIVE

회고 — 잘한 것과 의도적 선택의 트레이드오프

프로젝트를 마치며 두 갈래로 정리. **무엇을 잘했고, 어떤 부분이 트레이드오프를 따진 의도적 선택이었는지**. 각 선택의 한계와 다음 단계는 항목별 **→ 라인**에 함께 적음.

✅ 잘한 것

> 단순 CRUD를 넘어 **이벤트 유실(Outbox) · 최적화(VRPTW) · 동시성(비관적 락)** 등 운영 수준 문제를 **비즈니스 근거에서 출발해** 직접 설계. 스터디에서 학습한 락 트레이드오프를 실제 적용. "검증=동기, 통지=비동기"의 통신 의도 분리가 일관되게 관철됨.

🎯 의도적 선택

트레이드오프를 따져보고 현 규모에 적절하다 판단한 부분

배송 컨텍스트를 한 서비스로 통합 (delivery + manager + route)

초기 설계엔 `delivery / delivery_manager / delivery_route` 3개 서비스로 분할했으나, 실제 도메인 분석 결과 **3개 도메인이 동일 트랜잭션 경계에서 함께 변경**되는 강한 응집성을 보임 → 별도 서비스로 분리하면 분산 트랜잭션 비용만 증가한다고 판단해 **delivery-service 한 컨텍스트로 통합 + 내부 도메인으로 분리**.

→ "같은 트랜잭션에서 변하는 것은 같은 서비스로" — DDD의 애그리거트 경계 원칙을 실제 적용한 사례

Outbox 스케줄러가 단일 인스턴스 전제

분산 스케줄러·ShedLock·리더 선출 등 옵션을 검토했으나, **각 마이크로서비스가 단일 인스턴스로 배포되는 현 토폴로지**에선 스케줄러 중복 실행이 발생 X → 분산 락 도입은 오버엔지니어링.

→ 수평 확장 시점엔 ShedLock(DB 락) 또는 컨슈머 측 멱등 처리(이벤트 ID dedup) 추가 예정

full Saga 대신 Outbox 채택

다중 서비스 트랜잭션엔 Saga(보상 트랜잭션)도 후보였으나, 우리 케이스는 **단방향 통지·응답 불필요**라 full Saga의 복잡도가 과함.

→ 다중 단계 트랜잭션이 필요한 도메인에선 Saga가 정답 — 알고 있음

01 · OVERVIEW

LLM을 정량 평가하고, 안전하게 운영하는 백엔드

모놀리식 → MSA → LLMOps로 이어진 설계 성장의 마지막 매듭. **RAG 답변 품질을 LLM-as-a-Judge로 정량 평가**하고, **Prometheus·Loki·Claude를 묶어 알림에 원인 분석과 복구 액션까지 자동 부착**하는 시스템을 설계·구현했습니다. AI를 외부 API처럼 덧붙인 것이 아니라, **운영 모니터링 시스템 그 자체로 녹여낸** 것이 핵심입니다.

> "LLM을 검증 가능하게(Eval/Pairwise), 운영을 안전하게(HITL·다층 방어) 만든다."
>
> 모의투자 플랫폼에서 도메인 사실성이 핵심인 RAG 챗봇 + LLM 기반 AIOps + 측정 가능한 관측 스택을 주도 설계.
>
> ● 2026 · 부트캠프 5인 ● 담당: assistant · notification · monitoring 전담
>
> ● Java 17 · Spring AI 1.0 · pgvector · Kafka · GCP

2.38 → 4.04
faithfulness 향상
RAG 모델 교체 (LLM-as-a-Judge 점수 기반)

64.3% → 8.3%
환각률 −56pp
faithfulness < 3 비율

< 1분
MTTD
ServiceDown: scrape 15s × for 1m

74% ↓
캐시 히트 응답시간
3.68s → 0.95s (3.9배) · LLM 생성 호출 스킵

02 · SERVICE OVERVIEW

ants-camp — 개미배움캠프 · 가상 주식 투자 대회 플랫폼

실제 주식 데이터를 기반으로, ***가상 자산으로 투자 대회에 참여****하고* ***수익률 경쟁으로 순위를 겨루는*** *모의 투자 플랫폼.*

사용자는 한국투자증권 API로 가져온 **실시간 시세**를 보며 모의 매매를 진행하고, 매니저가 만든 대회 규칙(거래 시간·종목 제한·시드머니 등)에 따라 전략 경쟁을 합니다. 대회 종료 시점의 평가 자산 기반 **수익률로 순위·티어를 산정**하며, 단순 모의 투자를 넘어 **실시간 랭킹 · AI 챗봇 · LLM 기반 운영 알림 · MSA 확장 구조**까지 실제 서비스 수준의 경험을 목표로 설계했습니다.

서비스 6대 핵심 기능

01 · AUTH
회원 · 권한 관리
JWT 인증 + 사용자/매니저/관리자 RBAC. 매니저는 관리자가 등록한 계정으로만 활동.

02 · TRADING
가상 주식 매매
TradingView 실시간 조회 + 한국투자 API 기준가로 시장가 즉시 체결. 대회별 거래 시간·종목 제한 검증.

03 · COMPETITION
투자 대회 시스템
매니저 주도 대회 생성. 거래 규칙·참가 인원·시드머니 설정. 한 사용자가 여러 대회 동시 참가 가능.

04 · RANKING
실시간 랭킹 · 티어
주기적 수익률 계산 + Redis 캐싱. 종료 시 1·2·3등 발표 + TOP 10% 단위 티어 자동 산정.

05 · AI ASSISTANT ★ 담당
RAG 기반 챗봇 + 품질 평가
FAQ·약관·매매규칙을 벡터화해 의미 검색. LLM-as-a-Judge로 답변 품질 정량 측정.

06 · NOTIFICATION ★ 담당
통합 알림 + 모니터링
메트릭·로그 자동 수집 + LLM RCA → Slack HITL 액션. 중복 알림 SUPPRESSED 처리.

03 · TECH SELECTION

왜 PGAL Stack인가 — 모니터링 도구 선택 근거

MSA 환경의 핵심 비기능 요구는 **장애 추적·실시간 메트릭·중앙 집중 로그·자동 알림**입니다. ELK · CloudWatch · Datadog 등 후보를 **운영 비용 · 의존성 · 우리 규모 적합성** 세 축으로 비교한 결과, **Prometheus + Grafana + Alertmanager + Loki + Promtail**(이하 PGAL)이 적절하다고 판단했습니다.

ELK Stack
Elasticsearch + Logstash + Kibana
✓ 강력한 Full-text 검색 · 대규모 로그 분석 · Kibana 시각화 우수
✗ Elasticsearch 메모리 사용량이 매우 큼 · Logstash 리소스 무거움 · 운영 복잡도 높음
판단: 우리는 Kubernetes 규모의 초대형 시스템이 아니고, 로그 분석보다 **장애 추적이 더 큰 목적**. Docker 기반 경량 MSA엔 과한 구성.
REJECTED (over-spec)

CloudWatch
AWS managed
✓ AWS 서비스와 높은 통합성 · 별도 서버 운영 불필요 · 자동 스케일링
✗ AWS 의존성 증가 · 로그 저장 비용 누적 · 멀티 클라우드 대응 어려움
판단: 본 프로젝트는 **GCP + 로컬 Docker 기반**으로 운영 — 특정 클라우드 종속 회피.
REJECTED (vendor lock-in)

Datadog
SaaS APM
✓ SaaS 구축 쉬움 · UI/UX 우수 · 로그+메트릭+트레이싱 통합 APM
✗ 비용이 높음 · 데이터량 증가 시 과금 부담 · SaaS 의존성
판단: 학습/포트폴리오 환경에서는 **운영 비용 최소화가 중요**해 부적합.
REJECTED (cost-prohibitive)

PGAL Stack
Prometheus + Grafana + Alertmanager + Loki + Promtail
✓ MSA 사실상 표준(Prometheus) · Spring Boot Actuator 연동 용이 · ELK 대비 경량(Loki) · Grafana 단일 대시보드로 메트릭+로그 통합 · Docker SD로 컨테이너 자동 발견 · 전부 오픈소스로 비용 0
판단: 우리 규모(Docker 기반 MSA · 5인 팀 · 학습/데모 환경)에 정확히 맞는 균형. **운영 가시성을 확보하면서도 운영 부담을 최소화**.
CHOSEN ★

5개 도구 각각의 역할

- **Prometheus** 시계열 메트릭 scrape 15s
- **Grafana** 통합 대시보드 IaC provisioning
- **Alertmanager** 조건 기반 알림 Slack 라우팅
- **Loki** 경량 로그 저장 라벨 인덱싱
- **Promtail** 컨테이너 로그 Docker SD

도구	선택 이유 (한 줄)	활용 목적
Prometheus	MSA 사실상 표준 · Spring Boot Actuator 연동 용이 · 시계열 저장 최적화 · PromQL	CPU/Memory/API 응답시간/에러율/요청수 추적
Grafana	다양한 데이터 소스 + Loki/Prometheus와 높은 호환 + 대시보드 자유도	서비스별 메트릭 시각화 · 운영 대시보드 · 장애 실시간 확인
Alertmanager	Prometheus 자연스러운 통합 · 조건 기반 · Slack 연동 용이	임계치 초과 → Slack 알림 · API Error Rate 증가 감지
Loki	Elasticsearch 없이 로그 저장 · 라벨 인덱싱으로 비용 절감 · Docker 궁합	서비스별 중앙 로그 · 장애 로그 분석 · Trace ID 요청 추적
Promtail	Docker 로그 수집 단순 · Loki 공식 연동 · 설정 단순	컨테이너 로그 수집 · 서비스 라벨링 · Loki로 전달

04 · DESIGN GOALS

설계 도전 과제와 핵심 기여

핵심 도전 과제

- RAG 답변의 **사실성(faithfulness)** 정량 측정 — "느낌"이 아닌 **점수·승률**로 모델 결정
- LLM 평가의 2대 편향 차단 — **self-preference bias**(Judge가 자기 출력에 후한 경향) + **position bias**(Pairwise A/B 위치에 따른 편향)를 코드 레벨에서 모두 차단
- 단순 알림을 **"원인 + 권장 조치"**로 격상 — 운영자 컨텍스트 수집 단계 제거
- LLM 자동 실행의 위험 — 인프라/PII/위변조의 **다층 안전장치** 필요
- 토이 프로젝트의 한계 — **측정 가능한 관측 시스템** 자체를 차별점으로

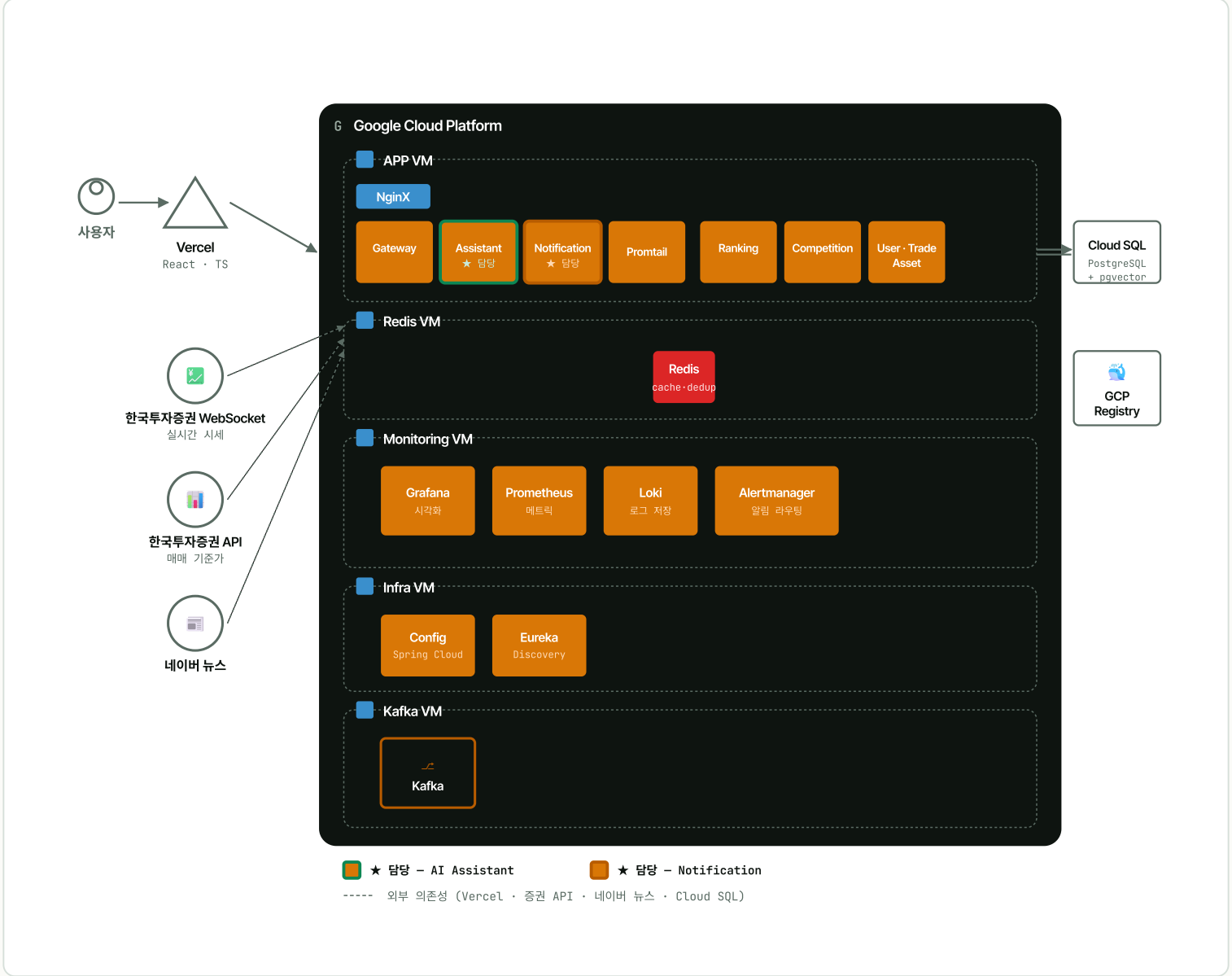
핵심 기여 (개인)

- RAG 파이프라인 + **LLM-as-a-Judge 3축 채점 + Pairwise** 시스템
- Self-preference + Position bias **코드 레벨 차단** + Judge 2종 교차
- Alertmanager → Redis dedup → Claude RCA → Slack **HITL** 파이프라인
- Defense-in-Depth: **인프라 보호 2중 차단 + HMAC-SHA256 + PII 마스킹**
- 관측 스택 주도 구축 — Prometheus/Loki/Promtail/Grafana/Alertmanager + alert 룰 **7종 `for:` 차등**
- RAG 응답 **pgvector 의미 캐시** — 히트 시 latency **74%↓** · LLM 호출 스킵

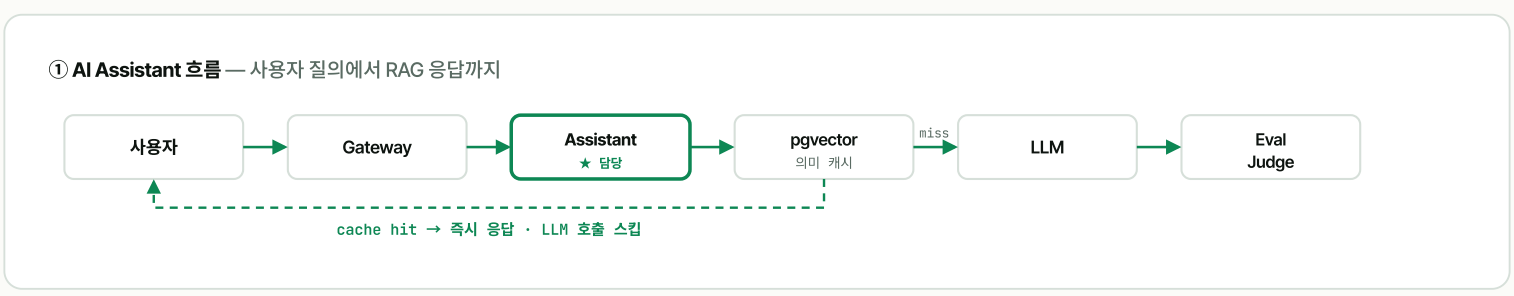
05 · ARCHITECTURE

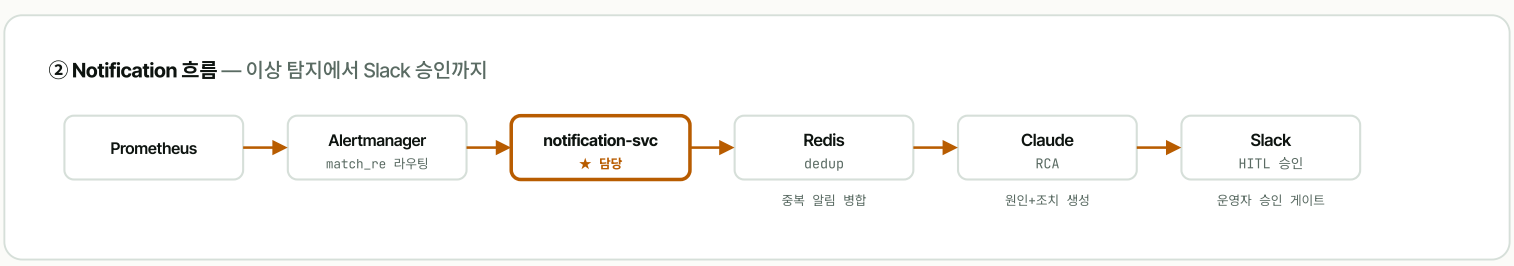
TO-BE 아키텍처 — GCP 5-VM 분리 토폴로지

Google Cloud Platform 위에 **역할별 VM 5개 그룹**으로 분리 배포합니다. **APP VM**(서비스 컨테이너) · **Redis VM**(랭킹 캐시·dedup) · **Monitoring VM**(관측 스택 단독) · **Infra VM**(Config·Eureka) · **Kafka VM**(이벤트). 외부 의존성(**Vercel** 프론트 · **한국투자증권 API/WebSocket** · **네이버 뉴스** · **Cloud SQL** · **GCP Container Registry**)은 GCP 경계 바깥에서 통신. AI Assistant(녹색)와 Notification(주황) 두 흐름이 동일 인프라를 공유합니다.



역할별 VM 토폴로지. 두 흐름의 단계별 상세는 아래 시퀀스 다이어그램 참고.


① **AI Assistant 흐름** — 사용자 질의에서 RAG 응답까지


② **Notification 흐름** — 이상 탐지에서 Slack 승인까지

설계 의사결정과 근거

결정	대안	선택 이유 — 정량 가정 · 확장 계획 포함
역할별 VM 5개 분리	단일 VM 통합 · K8s 클러스터	학습/포트폴리오 환경엔 K8s가 과함. 단일 VM은 SPOF·자원 경합 위험. **역할별 분리로 장애 격리와 자원 독립성 확보**. 운영 시 GKE로 단계적 이행 가능.
pgvector	Pinecone · Milvus	문서 수 수천 건 · QPS 한 자릿수 가정엔 cosine + IVFFlat로 p95 충분. **수십만 건 또는 멀티테넌시** 시점에 외부화 검토.
Spring AI + 모델명 라우팅	provider별 SDK 직접 호출	LLM(OpenAI/Claude/Gemini)을 Port 뒤로 추상화 → **모델 교체 시 application 무수정**. Pairwise 비교 실험 자유로움.
Claude Haiku (장애 분석)	Opus · Sonnet	알림은 빈도 높고 빠른 요약이 핵심 → **Opus 대비 약 1/10 비용**, max_tokens 2000.
Human-in-the-Loop	LLM 전자동 실행	운영 컨테이너를 건드림 → **AI는 추천까지, 실행은 사람 클릭**. 자동화 효율과 인간 판단 안정성 사이의 균형.
Reconciler(@Scheduled)	메시지 큐 (DLQ)	인제스트 동시성 분당 수십 건 가정엔 60초 주기 보정으로 SLA 만족. **분당 수백 건 이상** 시 Kafka DLQ 전환.
단일 인스턴스 → 분산 락 생략	ShedLock · 리더 선출	각 서비스 단일 인스턴스 배포 토폴로지 → 스케줄러 중복 실행 자체 발생 X. **수평 확장 시** ShedLock + 컨슈머 멱등 처리.

06 · KEY CONTRIBUTION ①

CONTRIBUTION #1

LLM 답변 품질을 "감"이 아닌 "점수"로 검증 — 정량 평가 파이프라인

모델 교체 결정의 정량적 근거 확보. faithfulness 2.38 → 4.04, 환각률 64.3% → 8.3%, Pairwise 6승 0패

P · 대다수 RAG 프로젝트는 답변 품질을 "느낌"으로 판단하고 모델 교체를 "감"으로 결정 → 품질 회귀(regression) 위험을 막을 정량 체계 필요. 특히 LLM 평가의 가장 알려진 함정 **self-preference bias**(LLM이 자기 출력에 더 후한 경향, 연구로 보고됨)가 차단되지 않으면 평가 결과 자체가 신뢰 불가.

A · **검색 → 채점 → 비교** 3단계로 정량 평가 파이프라인 구성. self-preference bias는 **코드 레벨에서 원천 차단** + Judge 2종 교차 검증으로 다중 방어.

> **검색** — pgvector 의미검색(topK×3 후 유사도 0.3·score-gap 0.8 컷오프 → 최종 topK=5)
>
> **채점** — Relevance / Faithfulness / ContextPrecision **3축 점수화**
>
> **비교** — Pairwise A/B 위치 교차(앞쪽 편향 회피)
>
> **bias 방어** — Judge==생성모델이면 채점 skip · Judge temperature 0 · Judge 2종(claude-sonnet-4-6 + gpt-4o) 교차 검증

R · 점수·승률 기반으로 **gpt-4o-mini → claude-haiku 교체**. **OpenAI 자사 gpt-4o조차 gpt-4o-mini를 낮게 평가** → 계열 편향 데이터로 반박.

CODE `EvalProcessor.java` · Judge 채점·bias skip `PairwiseProcessor.java` · A/B 위치 교차 `RetrievalReranker.java` · 검색 컷오프

★ 차별점 — 어떻게 차단했나 (3겹 방어)

① BIAS BLOCK
Self-preference 코드 레벨 차단
Judge 모델 == 응답 생성 모델이면 채점 자체를 **skip**. 자기 출력에 후한 경향을 원천 회피.

② CROSS
Judge 2종 교차 검증
`claude-sonnet-4-6 + gpt-4o` 양쪽 모두 돌려 한 모델에 결정 의존 X. **OpenAI 자사 gpt-4o조차 gpt-4o-mini를 낮게 평가.**

③ PAIRWISE
A/B 위치 교차 비교
A/B 위치를 **교차해 양방향 비교** → 위치 편향(앞쪽 유리)까지 회피. 결과: 6승 0패.

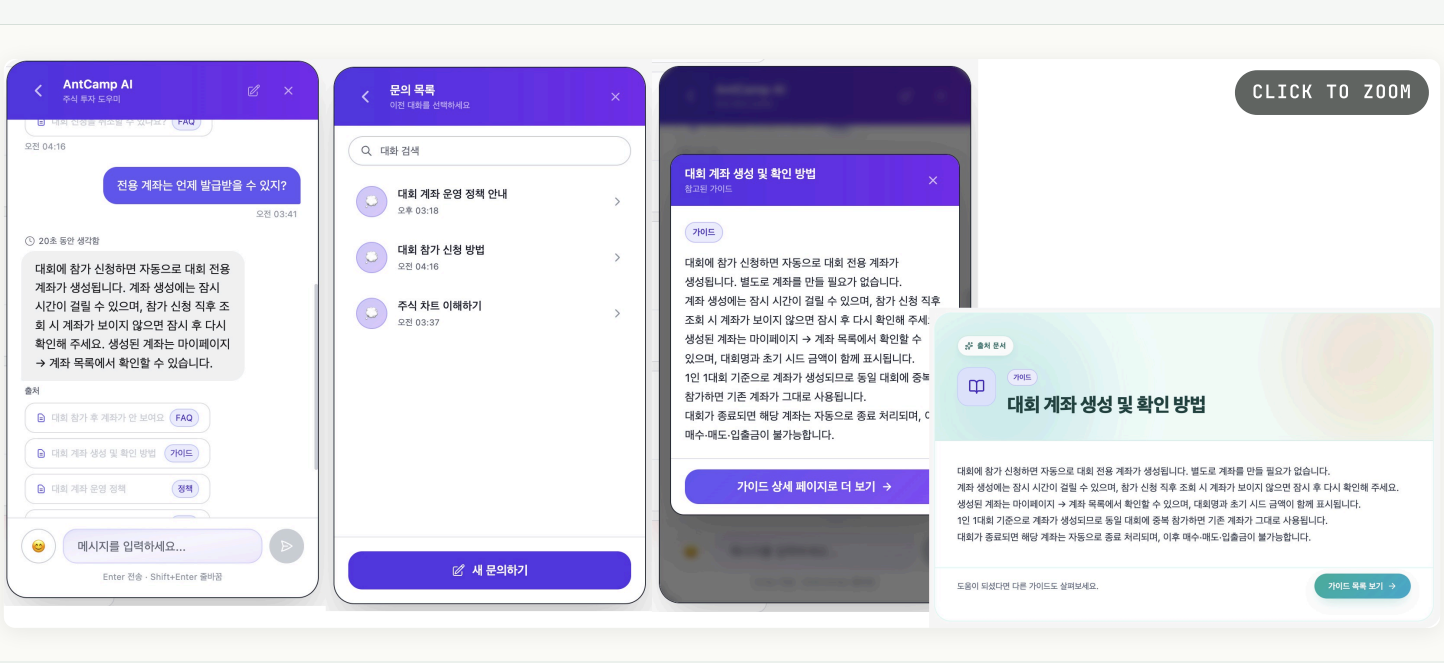
```
// EvalProcessor.judgeAndSave()
if (judgeModel.equalsIgnoreCase(ragQuery.getLlmModel())) {
    log.warn("Self-preference bias 방지: skip. model={}", judgeModel);
    return;
}
```

결과 — 데이터 기반 의사결정

지표	GPT-4O-MINI (N=28)	CLAUDE-HAIKU (N=12)
faithfulness	2.38	4.04
relevance	2.38	4.00
contextPrecision	2.32	3.92
환각률 (faith < 3)	64.3 %	8.3 %
Pairwise (A/B 교차)	0승	6승 0패 (TIE 0)

표본 40건은 통계적 검정력 충분치 않으나 ① 두 Judge 동일 방향 ② Pairwise 동일 방향 → 방향성 견고. 정밀화 시 골든셋 200건+ 확대 및 사람 평가자 inter-rater agreement 측정으로 Judge 신뢰도 자체 검증 계획.

평가 대상이 된 실제 RAG 챗봇 UI RAG · 출처 표시



위 정량 평가 시스템(LLM-as-a-Judge · Pairwise)의 평가 대상이 된 사용자 챗봇 화면. 단순 응답에 그치지 않고 **참조 문서의 출처를 함께 표시**해 사용자가 근거를 직접 확인할 수 있도록 설계했습니다.

- ✓ **질문 → 응답 → 출처 가이드** 3단계 표시 — "전용 계좌는 언제 발급받을 수 있죠?" 질문에 RAG로 검색한 가이드 문서 기반 답변 생성
- ✓ **출처 칩 클릭** → 우측 패널에서 참조한 가이드 문서의 핵심 내용 미리보기
- ✓ **"가이드 상세 페이지로 더 보기"** 버튼 → 전체 문서 페이지로 이동, 답변의 근거를 끝까지 추적 가능
- ✓ 세션 기반 대화 이력 관리 — 좌측 "문의 목록"에서 이전 대화 재조회 (세션 제목은 마지막 메시지로 자동 설정)

07 · KEY CONTRIBUTION ②

CONTRIBUTION #2

알림이 아니라 "원인 분석"을 전달 — LLM 기반 AIOps (Human-in-the-Loop)

MTTR 단축 + AI 자동화의 안전성을 사람 클릭으로 통제. 운영자의 컨텍스트 수집 단계 제거

P · Prometheus → Slack 단순 알림은 "임계값 초과" 사실만 전달 → 운영자가 Grafana·터미널을 왕복하며 원인 추적에 시간 소요(MTTR의 대부분). 동시에 LLM 자동 실행은 운영 컨테이너를 건드리므로 위험.

A · Alertmanager webhook부터 Slack 메시지까지의 파이프라인을 **dedup → 컨텍스트 수집 → LLM 분석 → HITL 액션** 4단계로 구성. **AI는 추천까지, 실행은 사람 클릭**이라는 안전선 확보.

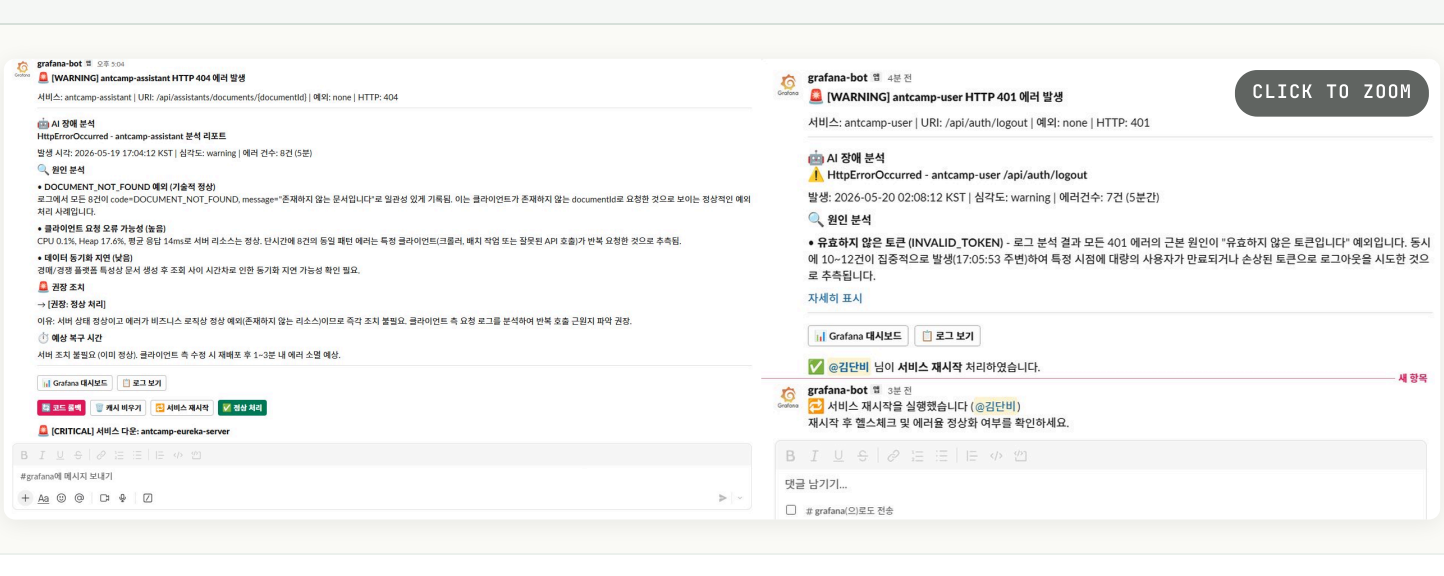
> **dedup** — Redis fingerprint(`SET NX` , TTL 1h)로 중복 알림 차단
>
> **컨텍스트 수집** — Prometheus(CPU/Heap/에러수/응답시간) + Loki(최근 10분 에러 로그) 자동 fetch
>
> **LLM 분석** — Claude(haiku)가 메트릭·로그 기반 RCA(원인 분석 + 권장 조치) 생성
>
> **HITL 액션** — Slack Block Kit으로 분석 + **복구 버튼 4종**(restart / rollback / cache-clear / false-alarm) 동시 노출

R · 알림 수신 후 수 초 내 "근거 메트릭/로그 + 권장 조치" 자동 게시. **AI를 외부 API가 아닌 운영 모니터링 시스템 그 자체로 녹임.**

```
// Slack 3초 응답 제약 → @Async ACK 후 메시지 갱신
@Async("slackActionExecutor")
public void executeAndNotifyAsync(SlackActionCommand command) {
    trySlack("처리 중", () -> alertPort.markAsProcessing(...));
    ActionResult result = executeAction(...);
    trySlack("완료/실패 갱신", () -> alertPort.markAsHandled(...));
}
```

CODE `NotificationApplicationService.java` · dedup→RCA→HITL `SlackBlockBuilder.java` · 복구 버튼 4종 `ClaudeApiClient.java` · LLM RCA `RedisDeduplicationAdapter.java` · SET NX dedup

실제 Slack 알림 + Human-in-the-Loop 액션 화면 HITL · 실제 운영



실제 Slack 채널에 자동 게시된 LLM 기반 장애 분석 메시지와 복구 버튼 클릭 후 스레드 갱신까지의 동작 흐름.

- ✓ **AI 장애 분석 리포트** — antcamp-user HTTP 400 에러 발생 시점, 심각도(warning), 에러율(7건/5분)을 자동 포착
- ✓ **원인 분석** — 1순위 클라이언트 입력값 검증, 2순위 일시적 클라이언트 트래픽, 3순위 서버 상태 등 **우선순위가 매겨진 분석** 제공
- ✓ **복구 버튼 4종**(빨강 박스 영역): 코드 롤백 · 캐시 비우기 · 서비스 재시작 · 정상 처리(false alarm) — 운영자가 상황에 맞게 선택
- ✓ **스레드 자동 갱신**(우측) — 버튼 클릭 시 `@Async` ACK 후 "처리중 → 완료/실패"로 메시지 갱신. Slack 3초 응답 제약 해소
- ✓ **Grafana 대시보드 / 로그 보기** 바로가기 버튼으로 운영자의 컨텍스트 전환 비용 최소화

08 · KEY CONTRIBUTION ③

CONTRIBUTION #3

LLM이 운영 시스템을 만지는 위험을 다층으로 차단 — Defense-in-depth

장애 시나리오 3종에서 fallback 동작 검증 완료

P · LLM 제안 시스템은 ① 인프라 서비스 오작동(LLM이 gateway/kafka에 destructive action 권유), ② 외부 위변조 요청(공개 endpoint 노출), ③ LLM 입력으로의 PII/시크릿 유출 등 다층 위험을 동시에 가짐.

A · 세 위험에 각각 독립적 방어선을 둠 (아래 3겹).

R · 세 시나리오 모두 fallback 동작 검증 완료 — ① notification-service 자기 장애 시 Alertmanager `match_re` fallback으로 Slack 직통, ② LLM API 장애 시 단순 텍스트 fallback, ③ 비동기 ingest 실패 시 Reconciler 자가복구.

CODE `SlackSignatureVerificationFilter.java` · HMAC·replay·timing `PromptUtil.java` · PII 마스킹 `NotificationProperties.java` · 인프라 화이트리스트

① INFRA PROTECTION
인프라 서비스 2중 차단
gateway·kafka 등 `infrastructureJobs` 는 SlackBlockBuilder에서 **버튼 자체 미노출** + executeAction 진입 시 **화이트리스트 차단**. API/payload 조작 우회까지 방어.
→ 두 겹 중 한 겹이 뚫려도 다음 겹이 막음

② SLACK FORGERY
위변조 · Replay · Timing 공격 방어
공개 endpoint에 전용 필터: **HMAC-SHA256 서명 검증** + timestamp 5분 초과 거부(replay) + `MessageDigest.isEqual` constant-time 비교(timing attack). body는 `CachedBodyRequest` 래핑.
→ 외부 위변조 요청 원천 차단

③ PII MASKING
LLM 입력 민감정보 마스킹
token · password · secret · bearer → `[REDACTED]` , 이메일 → `[EMAIL]` , 로그 4000자 제한. 외부 LLM 호출 직전 최종 검증.
→ 시크릿/PII 외부 API 유출 방지

09 · KEY CONTRIBUTION ④

CONTRIBUTION #4

측정 불가능한 운영을 측정 가능한 운영으로 — 관측 스택 주도 구축

MTTD < 1분 · alert 룰 7종 차등 설계 · 반복 알림 LLM 비용 0

P · 토이/부트캠프 프로젝트는 **측정 시스템 부재로 정량 수치를 못 남기는** 경우가 많음. *"측정 가능한 시스템을 처음부터 만들었다"*는 것 자체가 차별점이며, alert 룰 설계 역시 노이즈와 심각도를 함께 고려해야 함.

A · 관측 스택을 주도 구축하고 **alert 룰 7종을 `for:` 지속조건으로 차등 설계** — "터지면 큰가(심각도) × 자주 출렁이나(노이즈)"를 함께 반영.

> **수집** — Prometheus(15s scrape · 10 job) + Promtail(`docker_sd_configs` 컨테이너 자동 발견)
>
> **저장** — Loki(로그) · Prometheus TSDB(메트릭)
>
> **시각화** — Grafana provisioning IaC(대시보드 코드 관리)
>
> **알림** — Alertmanager + 룰 7종 차등 설계(아래 표)

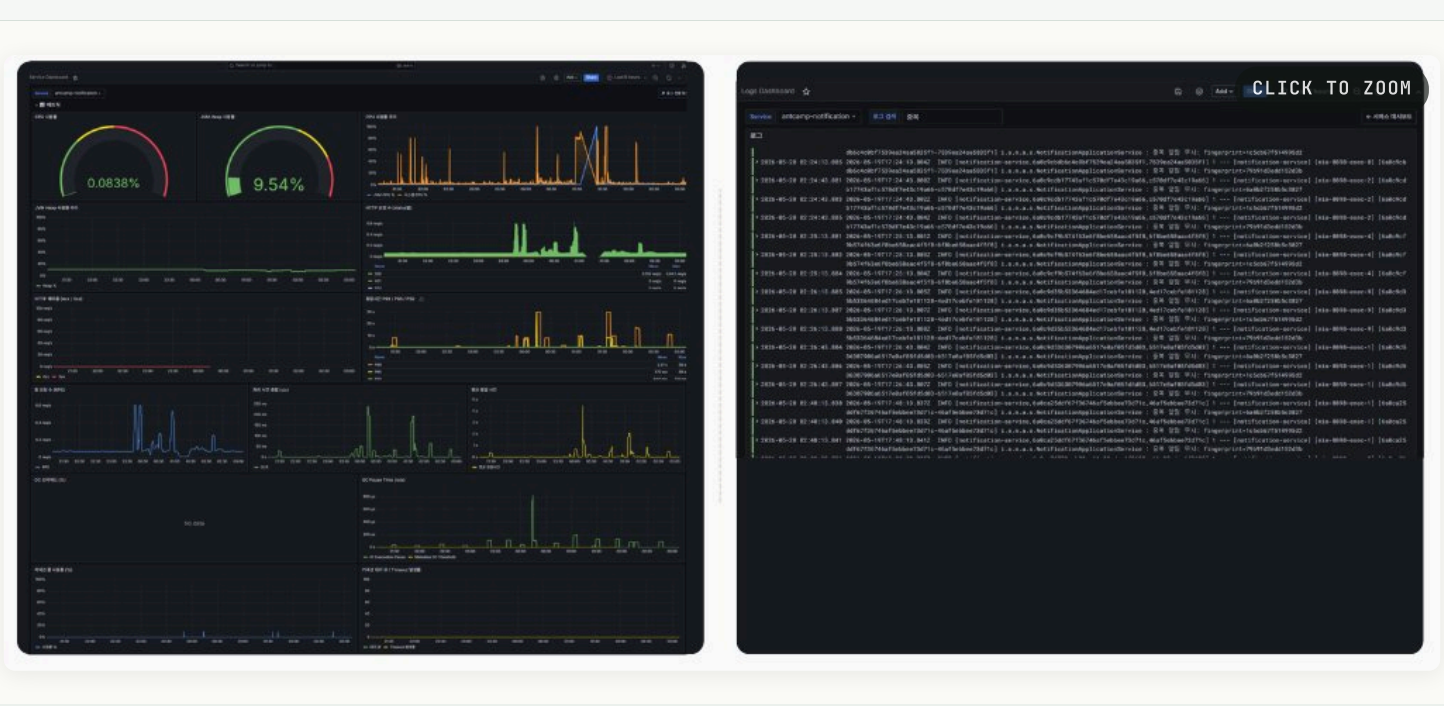
R · MTTD < 1분, 반복 알림 LLM 비용 0(dedup). SPOF 해소 fallback까지 완비.

CODE `alert-rules.yaml` · 룰 7종 for: 차등 `alertmanager.yaml` · match_re fallback `promtail.yaml` · docker_sd `monitoring/` · 스택 전체

룰	임계 / FOR	심각도	근거
ServiceDown	`up == 0 / 1m`	critical	scrape 15s × 4회 연속 실패라야 발동 → 순간 끊김과 구분
ConnPoolPending	`> 3 / 1m`	critical	대기 자체가 고갈 신호 → 가장 짧게
HighCPU	`0.8 / 2m`	critical	throttling 시작 구간, 100% 직전 여유
HighMemory · ConnPoolNear · HttpError	`/ 2m`	warning	스파이크와 추세 구분 가능한 지점
HighGcOverhead	`0.1 / 5m`	warning	단기 변동 잦아 가장 길게

★ **SPOF 해소** — notification-service 자기 장애 시 Alertmanager `match_re` (라벨 정규식 매칭으로 알림 라우트 분기) fallback으로 Slack 직통(LLM 분석 없는 단순 텍스트로 알림 보존).

실제 운영 중인 Grafana 통합 대시보드 관측 스택 · 실측



"측정 가능한 시스템을 만들었다"의 직접적 증거. 좌측은 **서비스 대시보드**(Prometheus 메트릭), 우측은 **로그 대시보드**(Loki 집계)로 **메트릭과 로그를 Grafana 단일 UI에서 통합**해 운영자의 컨텍스트 전환 비용을 제거했습니다.

- ✓ **핵심 게이지**(좌상단): CPU 사용률 0.0838% · JVM Heap 9.54% — 실시간 상태를 게이지로 즉시 가독
- ✓ **시계열 패널 다수**: CPU/메모리/HTTP 응답 시간/요청 수/스레드/GC/JVM CPU 사용률 등 **서비스 헬스의 다축 추적**
- ✓ **로그 통합 뷰**(우측): notification-service의 INFO/WARN 로그 실시간 스트림. **로그 라벨링**으로 컨테이너별·서비스별 필터링 가능

10 · KEY CONTRIBUTION ⑤

CONTRIBUTION #5

반복 질문엔 LLM을 다시 부르지 않는다 — pgvector 의미 캐시로 응답 latency 단축

캐시 히트 시 응답 0.95s — 미스(긴 답변) 대비 74%↓ · 3.9배 · LLM 호출 자체 스킵으로 지연·비용 동시 절감

P · RAG 답변은 LLM 생성에 수 초가 걸리고, FAQ·약관·매매규칙처럼 **동일·유사 질문이 반복**되는 도메인에선 매번 풀 RAG를 도는 게 지연·비용 낭비.

A · 첫 턴 질문을 임베딩해 **pgvector 의미 캐시**(`p_response_cache`)에서 `1 - cosine ≥ threshold` & 미만료면 저장된 답변을 즉시 반환, 미스 시 풀 RAG 후 캐시에 적재(TTL은 config 값).

> **의미 캐시** — 질문 임베딩 → 코사인 유사도 임계값 이상이면 히트 → **LLM 호출·벡터검색 생략**
>
> **적용 범위** — 첫 턴(히스토리 빈 상태)에서만 조회/저장 → 단발성 질문 트래픽에 최적
>
> **측정으로 검증** — hit율 · 클라이언트 wall-time · 서버 LLM `latencyMs` 를 워크로드별로 분리 측정

R · 캐시 히트 시 **0.95s**로 미스(긴 답변 3.68s) 대비 **2.73s↓ (74%, 3.9배)**, LLM 생성(2.41s) 자체를 제거.

CODE `ResponseCacheAdapter.java` · 의미 캐시 조회/적재 `RagApplicationService.java` · 첫 턴 캐시 흐름

측정 결과 — hit율 · latency

구분	클라이언트 WALL-TIME (중앙값)	서버 LLM LATENCYMS
미스 — 긴 답변(100–185자)	3.68s	2.41s
미스 — 짧은 거절(82자)	2.82s	1.34s
캐시 히트	0.95s	LLM 호출 스킵

워크로드	HITS/N	HIT율
exact-repeat (동일 질문 재요청)	10/10	100%
paraphrase (동일 의도, 다른 표현)	0/10	0%

11 · TROUBLESHOOTING

트러블슈팅 — 원인을 윗단까지 파고든 사례

"외부 호출 때문에 트랜잭션이 길어졌다"는 표면적 설명이 아닌, **커넥션 풀 → 톰캣 워커 스레드 → 세션 한계**까지 원인을 윗단으로 추적한 사례들.

#01 · LLM 호출 중 DB 커넥션 점유 → 풀 고갈 위험 트랜잭션 경계 분리

현상 · 가설	원인 분석 (WHY → WHY)	해결 & 검증
→RAG 답변 생성은 평균 수 초 소요 →유저메시지·봇메시지 저장을 단일 `@Transactional` 로 묶으면 **LLM 응답 대기 내내 HikariCP 커넥션 점유** →동시 요청 시 풀 고갈 → 톰캣 워커 스레드까지 블로킹 → 전체 타임아웃	→LLM 호출이 트랜잭션 안 → 커넥션 점유 →커넥션 풀 고갈 → 톰캣 스레드도 트랜잭션 잡은 채 블로킹 →스레드 풀 한계 도달 → 새 요청 거절 →풀 확장은 근본 해결 X — 요청량 늘면 다시 고갈	→**3구간 분리**: 유저메시지 저장(T1) → LLM 호출(**트랜잭션 없음**) → 봇메시지 + COMPLETED(T2) →omin 프로젝트 트랜잭션 경계 분리 교훈을 LLM 호출에 그대로 재사용 →*측정 후속 계획: Micrometer* `hikari_connections_active` *+* `llm_call_duration` *게이지로 점유 시간 정량화 예정*

CODE `RagApplicationService.java` · T1→LLM(무TX)→T2

#02 · 비동기 ingest 부분 실패 → DB · 벡터 불일치(orphan) 멱등성 · 자가복구

현상 · 가설	원인 분석	해결 & 검증
→문서 인제스트는 **청킹 → 임베딩 → 저장** 3단계 →임베딩 단계 실패 시 청크는 DB에 남고 벡터는 비어 orphan 발생 →재인제스트 시 단순 INSERT는 중복 벡터 생성	→다단계 비동기 처리 + 부분 실패 시점이 불특정 →재시도 시 멱등성 미보장 → 중복 가능 →실패 작업이 PROCESSING 상태로 영구 잔존 위험	→재인제스트 전 **기존 벡터 선삭제**(멱등성) → `DocumentReconciler` 60s 주기로 10분 PROCESSING 감지 →최대 3회 재시도 · PERMANENT 격리

CODE `DocumentReconciler.java` · 60s 보정·3회·PERMANENT

#03 · 동일 세션 동시 메시지 → seq 채번 race condition 비관적 락

현상 · 가설	원인 분석 — 대안 검토	해결 & 검증
→한 채팅 세션에서 빠르게 두 번 전송 →"최대 seq 조회 → +1 저장" 사이 **Read-Modify-Write race** →동일 seq 중복 발급 가능	→낙관적 락: 어차피 매번 충돌하는 채번엔 **재시도 비용만 누적** →분산 락(Redis): 단일 인스턴스 배포 토폴로지엔 과함 →채번은 **충돌 잦고 임계구역 짧음** → 비관적 락이 최적	→ `findMaxSeqForUpdate` 에 `@Lock(PESSIMISTIC_WRITE)` 로 채번 구간 보호 →재시도 로직 없이 **DB 레벨 원천 차단** →경력 race condition → 스터디 락 발표 → 본 프로젝트 적용. **3단계 학습 곡선의 매듭**

CODE `JpaChatMessageRepository.java` · findMaxSeqForUpdate

12 · STACK

기술 스택

영역	스택
Backend	Java 17 · Spring Boot 3.5 · Spring Cloud (Eureka · Config · Gateway) · **Spring AI 1.0**
AI / LLM	**LLM-as-a-Judge · Pairwise · pgvector** · Claude (Anthropic) · OpenAI · Gemini (fallback)
Data	PostgreSQL + pgvector · Redis · Kafka
Observability	Prometheus · Grafana · Loki · Promtail · Alertmanager · Zipkin
Infra	Docker · Terraform · GCP · GitHub Actions

13 · RETROSPECTIVE

회고 — 의도적 선택과 진짜 아쉬운 점을 구분하다

프로젝트를 마치며 세 갈래로 정리. **무엇을 잘했고, 어떤 부분이 의도적 선택이었으며, 어디가 진짜 아쉬웠는지**.

✓ 잘한 것

> LLM을 "붙이기"가 아니라 **검증 가능한 시스템**(Eval/Pairwise · bias 차단)으로, 운영을 **안전하게**(Human-in-the-Loop · 다층 방어 · 자가복구) 만든 것. 그리고 토이 프로젝트의 한계인 "측정 수치 부재"를 **관측 스택 주도 구축**으로 정면 돌파.

의도적 선택

트레이드오프를 따져보고 현 규모에 적절하다 판단한 부분

복구 액션이 단일 호스트 Docker socket 1:1
SSM · K8s rolling restart로 외부화하는 옵션도 검토했으나, **단일 호스트 데모 환경**에선 직접 호출이 더 단순·명료. 운영 복잡도 증가의 비용이 이득을 넘어선다고 판단.
→ 다중 인스턴스 시점엔 SSM/K8s API로 외부화 예정

Loki `replication_factor=1` · retention 미설정
단일 VM 토폴로지에선 HA가 의미 없음 → 단순 구성 선택. 데이터 영속성도 데모 범위 안에서 충분.
→ 운영 환경 전제로는 다중 노드 + 보관 정책 도입

인증을 게이트웨이 헤더에 위임
모든 서비스에 JWT 검증을 중복 구현 vs 게이트웨이 단일 검증의 단순함을 비교한 결과 **부트캠프 규모엔 후자가 적절**.
→ 운영 환경엔 서비스 레벨 JWT 한 겹 추가가 정석 — 알고 있고 다음에는 그렇게 설계

분산 락 생략
각 서비스가 단일 인스턴스 배포 토폴로지에선 스케줄러 중복 실행 자체가 발생 X → ShedLock 도입은 오버엔지니어링.
→ 수평 확장 시점에 ShedLock + 컨슈머 멱등 처리 추가

진짜 아쉬운 것

더 챙겼어야 하는 부분

의미 캐시 임계값이 엄격 — 유효 hit율 한계
측정해 보니 코사인 ≈0.95 임계값이 사실상 **동일 문구만 통과**시켜, 같은 의도라도 표현이 다른 질문은 전부 미스(paraphrase hit율 0%). 캐시 히트도 ~0.95s 소요(질문 임베딩 왕복 1회). 측정 자체도 단일 클라이언트·소표본 기준이라 동시성/부하 하 수치는 아님.
→ 임계값 0.90~0.92 완화(미묘하게 다른 질문에 오반환 위험과 트레이드오프) · 정규화 질문 exact-match 단축 경로 · 임베딩 캐싱 검토